MSc Data Science

Machine Learning Coursework

Chamika Deshan
COMScDS251P-015

Setup

For this project we used python for programming, and incoperated clean architecure and dependency injection for seperation in concern. The source code can be downloaded from git repositoy below.

https://github.com/deshancha/machinelearning_cw

Adjsuting the paramter below in .env file can Disable/Enable Logs, Default is enabled.

LOG_ENABLED=1

Q1: Applied Supervised Machine Learning

Task 1 : Data source selection and Supporting Factors

UCI Adult Income Dataset

- ◊ From UCI ML repository which is reliable and well cited.
- ◊ Dataset contains 48,842 records (>15000), divided into 32561 training and 16281 testing records.
- ◊ Has both numerical and categorical features, with 14 predictor variables.

  Numerical

  1. Age
  2. Final Weight (Individuals with similar demographic characteristics receive similar weights)
  3. Education Num (Total number of years of education)
  4. Capital Gain
  5. Capital Loss
  6. Hours per week

  Categorical

  1. Work class
  2. Education
  3. Marital status
  4. Occupation
  5. Relationship

6. Race
7. Sex
8. Native Country

◊ Support supervised learning, income exceed 50K or not

◊ Has anomalies (missing values, class imbalance)

Target variable and prediction objective

Target variable is the Income

This is a binary categorical variable and represents individual annul income level.

1. Less than or equal to 50,000: 76%
2. Greater than 50,000: 24%

Due to 3:1 class distribution as depicted in figure 1, which is a skewed dataset need to address class imbalance before feeding it to ML model.

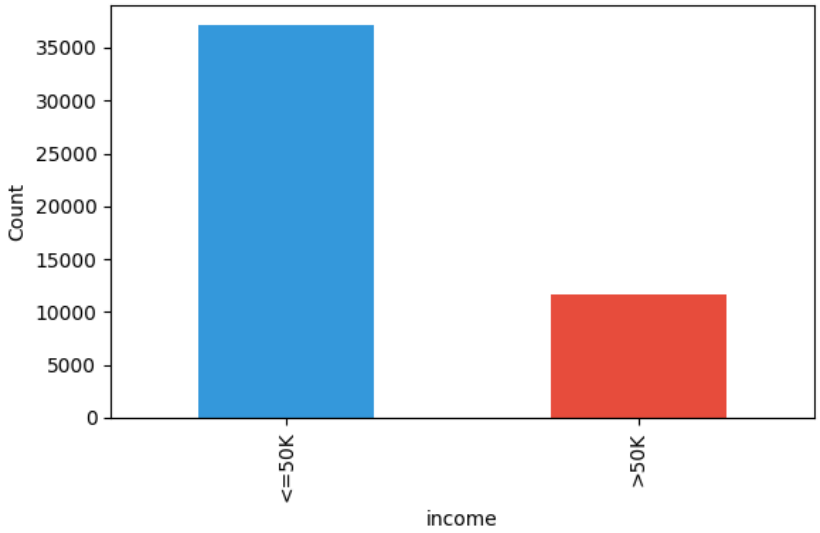


Figure 1: Income distribution

Creating supervised binary classification is the prediction objective

This uses demographic, Occupational, Personal and Financial features listed above.

Literature Review

1. **Scaling Up the Accuracy of Naive-Bayes Classifiers: A Decision-Tree Hybrid [1]**

   Source: Proceedings of the Second International Conference on Knowledge Discovery and Data Mining

Summary

Addresses the limitations of Naive Bayes and Decision Trees models. This proposes a framework NBTree to improve prediction accuracy on large, mixed features data.

2. **Comparative Study of Machine Learning Algorithms on Census Income Data Set [2]**

   Source: International Journal of Engineering Research and Applications

Summary

   a. Compares supervised classifiers on the UCI Adult income data.
   b. Testing with nonlinear models using accuracy and F1scores
   c. Shows gradient boosting has the highest predictive power for this dataset with F1-Score of 0.65
   d. Shows population weight has no linear correlation with individual income status.

3. **Machine Learning on UCI Adult Dataset Using Various Classifier Algorithms And Scaling Up The Accuracy Using Extreme Gradient Boosting [3]**

   Source: Wordpress Technical Report

Summary

   a. Presents multi model optimization strategy with greater accuracy on the Adult dataset.
   b. Incorporates classifications like kNN, Logistic Regression before building the multi model strategy

4. **Evaluation of Fairness in Machine Learning Models using the UCI Adult Dataset [4]**

   Source: Brazilian Symposium on Databases Data share Track

Summary

   a. Address trade offs between classification accuracy and algorithmic fairness
   b. Evaluate Regression algorithms and Ensemble algorithms with incoperating socio economic attributes like Sex and race

5. **Feature Selection for Financial Data Classification Using Random Forest, Boruta, and Recursive Feature Elimination [5]**

   Source: International Information and Engineering Technology Association

Summary

   a. Targets dimension reduction, feature selection and model interpretability on financial records using Adult income dataset
   b. Recursively eliminate features and bring down 14 features to optimized 5 features which minimizes the overfitting in classification and regression models

6. **Predicting Adult Income Utilizing Various Artificial Intelligence Models**

   Source: The Eurasia Proceedings of Science, Technology, Engineering and Mathematics

Summary

a. Testing accuracy of multi model systems using balanced and unbalanced datasets.
b. Random forest has good accuracy on the dataset

Research Gaps and Proposed Approach

1. **Lack of Model Family Comparison**

   Existing studies in the above-mentioned research papers focuses on single model or compare only two or three algorithms (such as Random Forest vs kNN). We would implement and **compare multiple machine learning algorithms** in different categories, such as Linear Model (Logistic Regression), Distance-Based Model (KNN), Tree-Based Model (Random Forest), Ensemble Boosting Model (XGBoost), and Deep Learning (ANN).

2. **Generic Imbalance Handling**

   Research papers apply class imbalance methods without consideration of the specific model type. We would try **targeted imbalance strategy** for specific models. Fore example Scale Positive Weight for XGBost, Class Weighting for linear and tree-based.

3. MLOps

   Here we would implement containerized deployment with model versioning with monitorig strategies targeting **end to end MLOps pipeline**.

Task 2: Data Analysis and Feature Engineering

Exploratory Data Analysis (EDA)

Correlation Analysis

As depicted in figure 2 correlation matrix shows weak linear correlations between all numerical features. All correlation coefficients are less than 0.15. So, we keep all variables because they could bring more information. And **linear model alone could not work** well because of weak correlation between numerical variables. So, we need to bring tree-based models like random forest or XGBoost **because they can capture non-linear relationships**.

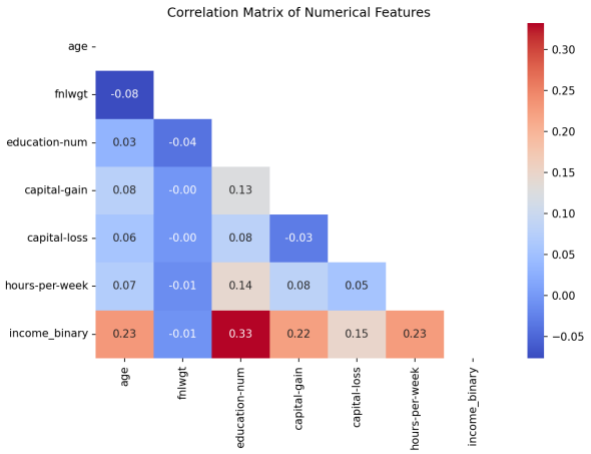


Figure 2: Correlation Matrix of Numerical Features

Numerical Feature Distribution

Figure 3 depicts histogram of numerical features.

- ◊ **Hours-Per-Week**: Hours per week centered around 40 hrs because that is standard week hours.
- ◊ **Education-Num**: Education represents years of education peaking at 9(Highschool Graduate).

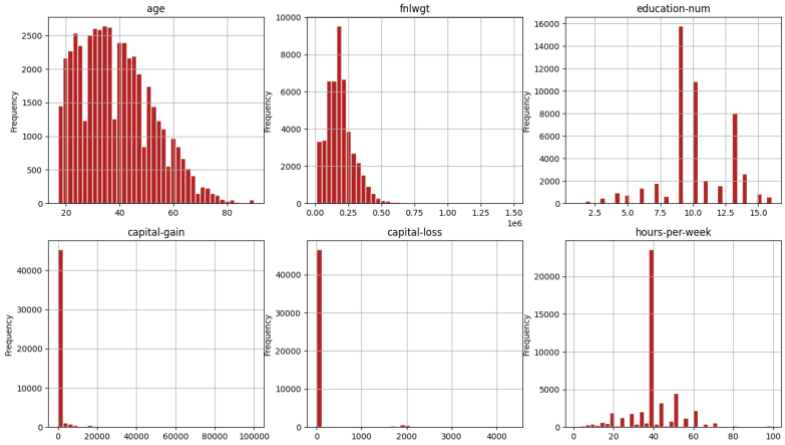


Figure 3: Numerical Features Distribution

Categorical Features vs Income

Figure 4 depicts categorical features vs Income percentage.

- ◊ **Marital-Status**: Individuals with Married-civ-spouse status have highest proportion of high income
- ◊ **Occupation**: Executive and Managerial (Exec-managerial) class and Professional Specialty (Prof-specialty) class has the highest high income

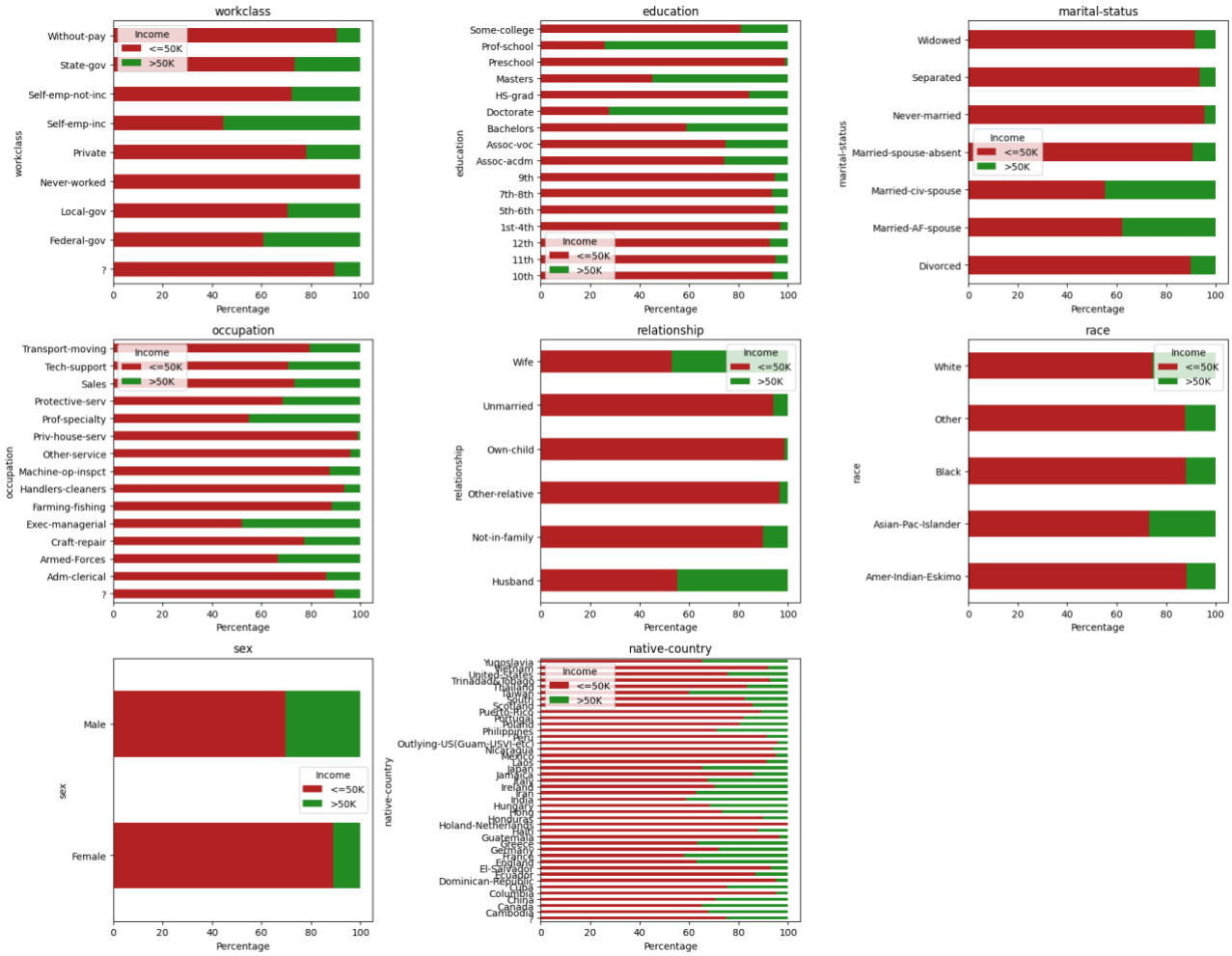


Figure 4: Categorical Income distribution percentage

Data Cleaning

Handling Missing Values

Missing values represented as '?' in the original dataset and we replaced with 'NaN' and the rows were dropped. Resulting dropping 3620 rows (~7%). This is acceptable because it still has 45175 records.

Removing Duplicates

Dropped 47 duplicate records.

Feature Elimination

Remove fnlwgt column

- Table 1 contains the correlation coefficient of numerical variables to the target 'Income' variable. And fnlwgt has coefficient of -0.0072 showing virtually no linear association.

Feature	Correlation Coefficient
fnlwgt	-0.007227
age	0.236839
education-num	0.332981

hours-per-week	0.227146
income_binary	1.000000

Table 1: Numerical to Income Correlation

- ◊ Above 2nd paper in literature review "Comparative Study of Machine Learning Algorithms on Census Income Data Set", they also removed fnlwgt from calcaultion.
- ◊ This is not an attribute of individual instead statistical calculation based on demographic characteristics.

Feature Engineering

We introduced five new features based on existing understanding of the features.

1. '**capital_net'** (capital-gain - capital-loss): Captures the net return of capital.
2. **'has_capital'** Binary feature (1 or 0) based on capital gain or capital loss respectively.
3. **'age_group'** Binned Life stages: Young (0-25), Early-Career (25-35), Mid-Career (35-45), Senior (45-55), Pre-Retire (55-65), Retired (65-100).
4. **'hours_category'** Binned hours-per-week: Part-Time (0-34), Full-Time (34-40), Overtime (40-100).
5. **'edu_group'** Simplified education: Grouped into Low, High-School, Some-College, Bachelors, Advanced.

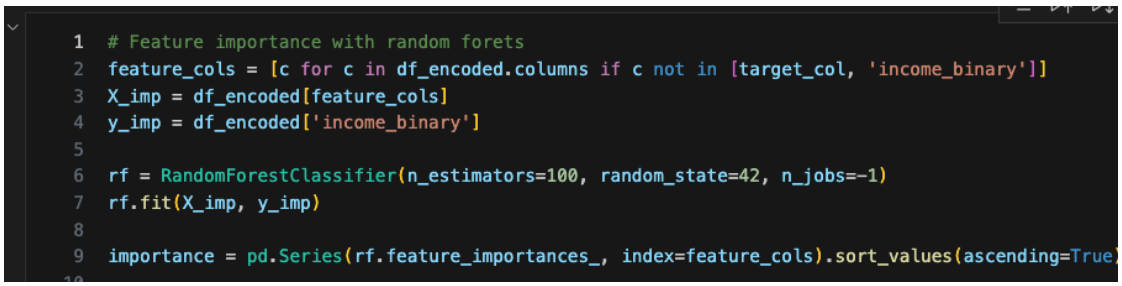
Feature Importance with Random Forest Classifier

We use random forest classifier here because unlike standard correlation which look at straight linear relationships random forest captures the nonlinear boundaries. Figure 5 depicts the python code.

Eg: Age does not have straight line relationship with income (income may increase at middle age but drops when they retire). Correlation would miss this but Random Forest catches this by threshold split (age > 25 and age < 25)

Below are the top 10 features sorted by importance

1. age: 17.15%
2. relationship: 11.32%
3. hours-per-week: 8.80%
4. occupation: 8.76%
5. capital_net: 7.93%
6. marital-status: 6.97%
7. capital-gain: 6.55%
8. education-num: 5.68%
9. edu_group: 5.36%
10. workclass: 5.06%



```python
# Feature importance with random forets
feature_cols = [c for c in df_encoded.columns if c not in [target_col, 'income_binary']]
X_imp = df_encoded[feature_cols]
y_imp = df_encoded['income_binary']

rf = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
rf.fit(X_imp, y_imp)

importance = pd.Series(rf.feature_importances_, index=feature_cols).sort_values(ascending=True)
```

Figure 5: Feature importance with Random Forest Classifier

Task 3: Model Development and Optimization

Algorithm Selection

Here we implemented five machine learning models representing disticty algorithm families.

1. **Logistic Regression (Linear Model):** This is fast to train, predicts True or False, and S shape logistic function for prediction.

2. **K-Nearest Neighbors (KNN) (Distance-based):** Classifies based on similarity in multi-dimensional feature space.

3. **Random Forest (Tree-based Ensemble):** Multiple decision trees. Natively handles non-linear boundaries and categorical features.

4. **XGBoost (Ensemble Boosting):** A gradient boosting machine. Builds Tress sequentially to correct mistakes. Can handle missing values natively.

5. **Artificial Neural Network / MLPClassifier (Deep Learning):** Stacks hidden layers with activation functions to learn complex, non-linear feature representations.

Preparing the Dataset for ML

We need to prepare the dataset with preprocessing and scaling to avoid crashing and improve accuracy. Here we would use 'ColumnTransform' to handle Numertical and Categorical features.

1. StandardScalar(Z-Score) for Numerical
2. OneHotEncoder for Categorical: Turns category labels to numbers where ML can understand

This also prevents data leakage where fit() the model it calculates only from training data and not touches test data which could fails the model in production.

```python
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), num_cols),
        ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False), cat_cols)
    ]
)
```

Figure 6: Preprocessing and Scaling

Handling Class Imbalance

Out of 5 main class imbalance techniques we would use three here for balance skewed data set before feeding to ML model.

- **Logistic Regression & Random Forest:** Using class weighting. We do not worry about balancing and tell model creation algorithm to balance it (class_weight='balanced').
- **XGBoost:** Scale Positive Weight, Here we give weight to the algorithm. Initially we identified imbalance ratio is 3.18. scale_pos_weight = 3.18
- **ANN:** Here we use RandomOverSampler. This would balance by duplicating existing in minority. Here the high income class(> 50K) is the minority. 3 out of every 4 rows network look will be low income(=<50K). Network would get 76% accuracy score by shifting weights to predict “low income” every time. Gradients for high income class become smaller. By Random Oversampling we can ensure that every training batch network finds equal high and low income data. This forces network to actively adjust its weight to learn the features.

Hyperparameter Optimization and Cross-Validation

```python
# ---------- Logistic Regression ----------
def _logistic_regression(self, X_train, y_train, preprocessor, model_dir, model_results):
    lr_pipeline = Pipeline([
        ('preprocessor', preprocessor),
        ('clf', LogisticRegression(class_weight='balanced', random_state=42))
    ])
    lr_param_dist = {'clf__C': [0.01, 0.1, 1.0, 10.0]}

    # n_jobs=-1 use all available cores
    # scoring='f1' for F1 score
    lr_search = RandomizedSearchCV(
        lr_pipeline, param_distributions=lr_param_dist, scoring='f1', n_jobs=-1, random_state=42
    )
    lr_search.fit(X_train, y_train)
    best_lr = lr_search.best_estimator_

    self.logger.info(f"Best parameters: {lr_search.best_params_}")
    self.logger.info(f"Best CV F1: {lr_search.best_score_:.4f}")

# ---------- KNN ----------
def _knn(self, X_train, y_train, preprocessor, model_dir, model_results):
    knn_pipeline = Pipeline([
        ('preprocessor', preprocessor),
        ('clf', KNeighborsClassifier())
    ])
    knn_param_dist = {
        'clf__n_neighbors': [5, 9, 15],
        'clf__weights': ['uniform', 'distance']
    }

# ---------- Random Forest ----------
def _random_forest(self, X_train, y_train, preprocessor, model_dir, model_results):
    rf_pipeline = Pipeline([
        ('preprocessor', preprocessor),
        ('clf', RandomForestClassifier(class_weight='balanced', random_state=42))
    ])
    rf_param_dist = {
        'clf__n_estimators': [100, 200],
        'clf__max_depth': [10, 20, None],
        'clf__min_samples_split': [2, 5]
    }
```

Figure 7: Minimum python code for Train and Best Hyper param, F1 calculation

Figure 7 depcits the minimized python code for model tuning and findinf best hyper param values and F1 scores. This code can find in use case under file name train_and_tune_usecase.py

We use Randomized Search Cross-Validation for finding best settings for hyperparameters. This would find the best settings with good enough accuracy without wasting computing time.

Considerations

True Positive = TP
True Negative = TN
False Positive = FP
False Negative = FN

Accuracy = Ratio of correct predictions to the total predictions

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision = Out of predicted positive how many actual positive.

$$Precision = \frac{TP}{TP + FP}$$

Recall(Sensitivity) = Out of actual positive how many successfully predicted positive.

$$Recall = \frac{TP}{TP + FN}$$

F1-Score = Balance between Precision and Recall

The fundamental goal is decreasing FP and FN values which eventually increase Precision and Recall values.

$$\text{F1} - \text{Score} = 2 \text{ x } \frac{\text{Precision x Recall}}{Precision + Recall}$$

F1-Score is the harmonic mean, according to above equation increasing both precision and recall would increase the F1-Score but we need to make sure make the model smarter with approaches like below rather than manually adjusting the threshold values.

1. More Features
2. Better Algorithms
3. Collect More data

Model Evaluation Results

Model	Hyperparameters Optimum	F1-Score
Logistic Regression	{'C': 1.0}	0.6957
KNN	{'weights': 'uniform', 'n_neighbors': 15 }	0. 6716
Random Forest	{'n_estimators': 20, 'min_samples_split': 5, 'max_depth': 20}	0. 7094
XGBoost	{'n_estimators': 100, 'max_depth': 7, 'learning_rate': 0.2}	**0.7205**
ANN	{'hidden_layer_sizes': (50,), 'alpha': 0.01, 'activation': tanh}	0.6936

Table 2: Model Cross Validation F1-Score and Optimum Hyprparameters

XGBoost achieved the highest cross validation F1-Score of **0. 7205**

Task 4: Performance Evaluation and Model Comparison

Evaluated 5 optimized models on the test set using Accuracy, Precision, Recall, F1 Score, ROC-AUC, and PR-AUC.

Model Performance Comparison

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC	PR-AUC
Log-Regression	81.43%	58.75%	84.20%	0.6921	0.9086	0.7811
KNN	83.75%	69.51%	61.38%	0.6520	0.8894	0.7478
Random Forest	83.21%	62.33%	81.86%	0.7067	0.9136	0.7956
XGBoost	83.56%	62.3%	85.36%	**0.7203**	**0.9265**	**0.8271**
ANN	82.01%	60.15%	81.38%	0.6917	0.9083	0.7789

Table 3: Model validation Summary

Considerations

ROC-AUC : Receiver Operating Characteristic – Area Under the Curve
This tells how good model separating High Income and Low-Income classes.

PR-AUC: Precision-Recall Area Under the Curve
Evaluates model performance across all possible decision thresholds. Here it can cope better with imbalance dataset as we have here over ROC-AUC because it ignores True Negative.

According to Table 3 we can highlight following

- ◊ XGBoost is the top performer with highest F1-Score(0.7023) which leads to **most balance between precision and recall**. ROC-AUC and PR-AUC also highest here so XGBoost can **separate the High and Low income better**.
- ◊ Both XGBoost and Logistic Regression is good in identifying high earners. They have the highest Recall values 85% and 84% respectively, so it has the minimum False Negative which minimizes predicted high earner but actually low.
- ◊ Random Forest offers best overall performance with good F1-Score of 0.7067 and ROC-AUC of 0.9136.

Visualization

ROC Curve

Figure 8 depicts ROC-Curves of the models and it shows XGBost has the highest AUC which achieves highest diagnostic capability. KNN has the lowest.

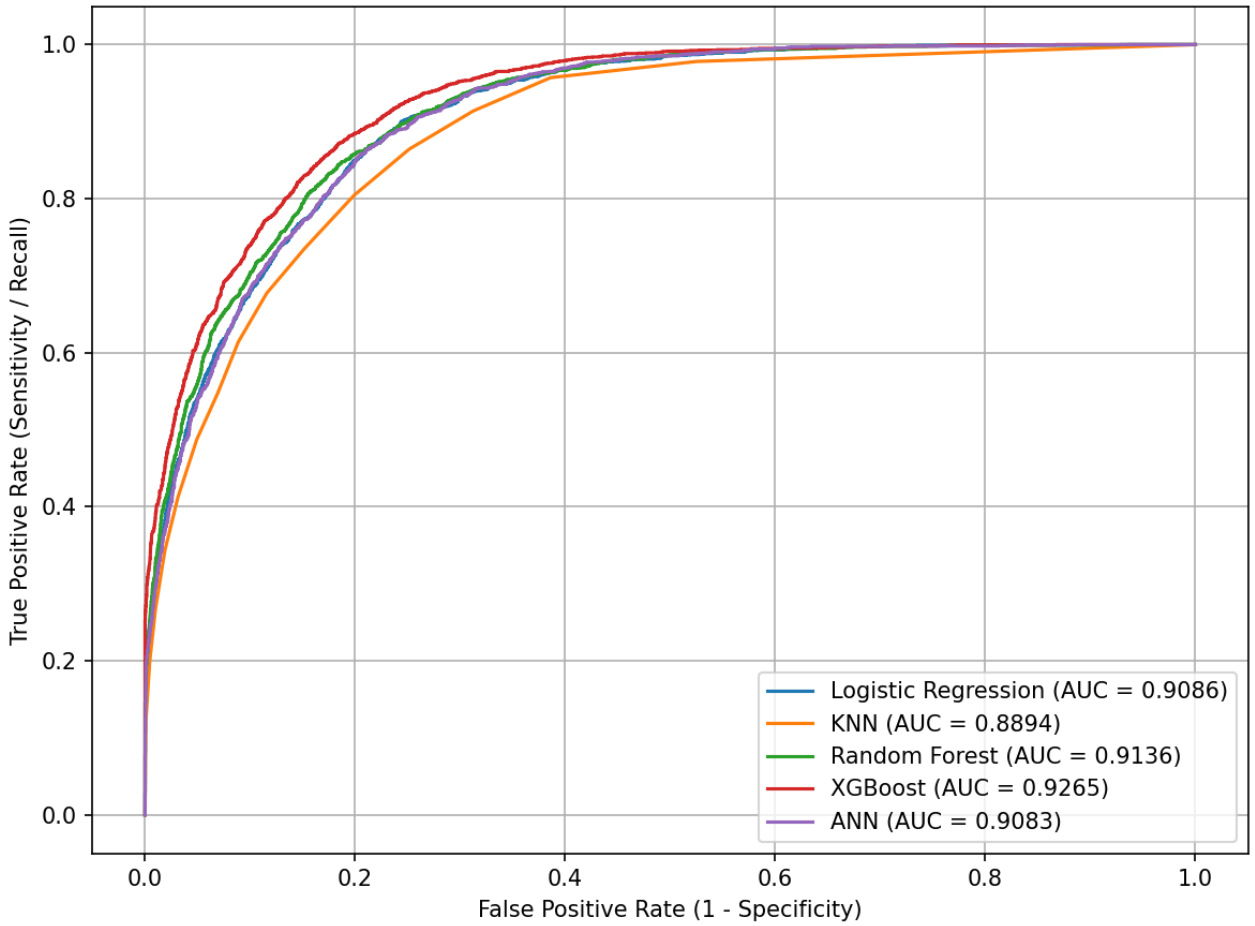


Figure 8: ROC Curves

Precision- Recall Curve

Figure 9 depicts PR Curves of the models and it shows XGBost has the highest average precision which is very good in finding high income earners.

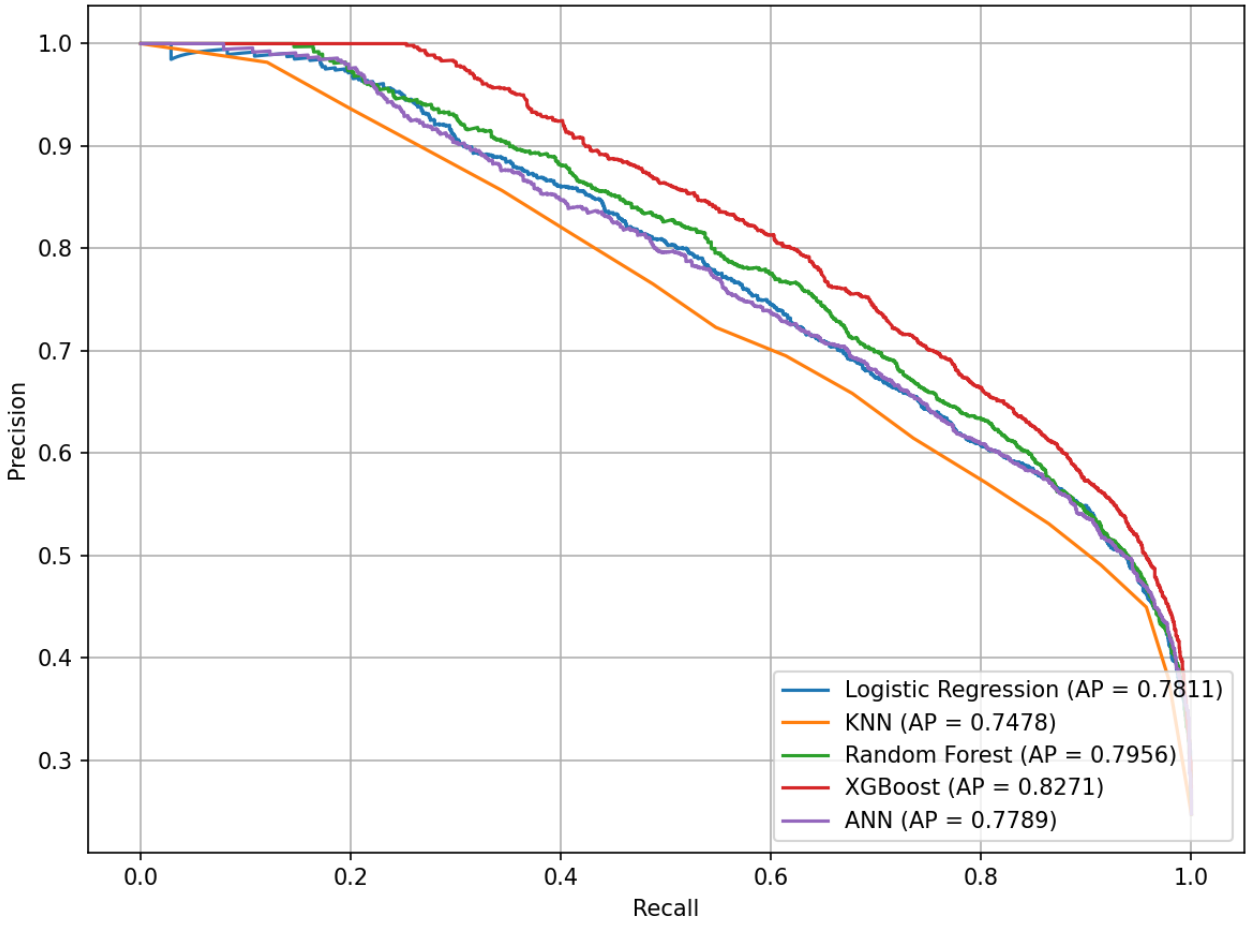


Figure 8: PR Curves

XGBoost Error Analysis

We analyzed the errors (False Positive and False Negative) of the best performing model which is XGBoost across Sex and Educational Group which had the top two highest correlation coefficient above.

False Positive (FP) : Predict > 50K(High Income) but in real its < 50K(Low Income)
False Negative (FN): Predict <50K(Low Income) but in real its >50K(High Income)

Sex	Total	Errors	Error Rate	FP	FN
Male	6093	1249	**20.50**	1006	243
Female	2942	236	8.02	151	85

Table 4: Error Profile By Sex

Table 4 depicts Gender based analysis and we can that model is more prone to False positive on Males, predicting them high earners more often than they really are.

Education Group	Total	Errors	Error Rate	FP	FN
Advanced	768	155	20.18	135	20
Bachelors	1512	302	19.97	238	64
College	2711	484	17.85	400	84
High-School	2915	474	16.26	364	110
Low	1129	70	6.20	20	50

Table 5: Error Profile By Education Group

According to Table 5 above its noticeable that Advanced and Bachelors degree has high eror rate (~20%). Predicting high income due to has degree but actual income is lower.

Low education group ha very low error rate (6.20%).

Task 6: Cloud Deployment and MLOps Strategy

Model Packaging and Containerization

To ensure runtime reproducibility, portability(local, AWS ECS), scalability and resolving easy deployment without worrying about dependency versions the machine learning model pipeline is packaged into an standalone environment using docker. Figure 9 shows docker file content.

Dockerfile

```
# Python base image
FROM python:3.10-slim

# Env vars
ENV PYTHONPATH=/app/src \
    PORT=8000

WORKDIR /app

# build dependencies
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    && rm -rf /var/lib/apt/lists/*

# Install Python requirements
COPY Q1/requirements.txt /app/
RUN pip install --no-cache-dir -r requirements.txt

# Copy source code
COPY Q1/src/ /app/src/
COPY Q1/app.py /app/

# Copy Git and DVC configurations to resolve model tracks inside the container
COPY .git /app/.git
COPY .dvc /app/.dvc
COPY Q1/outputs/models/xgboost_model.joblib.dvc /app/Q1/outputs/models/xgboost_model.joblib.dvc

# Expose port for FastAPI
EXPOSE 8000

# Start uvicorn for FastAPI
CMD ["uvicorn", "task_6.main:api", "--host", "0.0.0.0", "--port", "8000"]
```

Figure 9: Dockerfile

API Development

FastAPI were used to implement Restful API because it inherantly support asynchronous request handling and automatic swagger documentation. The request validation were done with Pydantic. Here api handles the feature engineering dynamically at server side. Figure 10, 11 amd 12 depicts the python codes for this part.

```python
class PredictionResponse(BaseModel):
    prediction: int = Field(..., description="0 for <=50K, 1 for >50K")
    label: str = Field(..., description="Income category label (<=50K or >50K)")
    probability: float = Field(..., description="Probability of income being >50K")

class BatchPredictionResponse(BaseModel):
    predictions: List[PredictionResponse]
```

Figure 10: Request Validation

```python
def _setup_routes(self) -> None:
    @self.api.post("/predict", response_model=PredictionResponse)
    def predict(input_data: PredictionInput):
        self.logger.info("Single prediction request received")
        try:
            results = self.usecase.execute([input_data])
            return results[0]
        except Exception as e:
            self.logger.error(f"Prediction error: {str(e)}")
            raise HTTPException(status_code=400, detail=f"Prediction error: {str(e)}")

    @self.api.post("/predict_batch", response_model=BatchPredictionResponse)
    def predict_batch(input_data: List[PredictionInput]):
        self.logger.info(f"Batch prediction request received for {len(input_data)} records.")
        try:
            results = self.usecase.execute(input_data)
            return BatchPredictionResponse(predictions=results)
        except Exception as e:
            self.logger.error(f"Batch prediction error: {str(e)}")
            raise HTTPException(status_code=400, detail=f"Batch prediction error: {str(e)}")
```

Figure 11: API Endpoints for realk time and batch prediction

```python
raw_data = [item.model_dump(by_alias=True) for item in inputs]
df = pd.DataFrame(raw_data)

# Feature Eng
df['capital_net'] = df['capital-gain'] - df['capital-loss']

df['has_capital'] = ((df['capital-gain'] > 0) | (df['capital-loss'] > 0)).astype(int)

df['age_group'] = pd.cut(df['age'],
                         bins=[0, 25, 35, 45, 55, 65, 100],
                         labels=['Young', 'Early-Career', 'Mid-Career',
                                 'Senior', 'Pre-Retire', 'Retired'])

df['hours_category'] = pd.cut(df['hours-per-week'],
                              bins=[0, 34, 40, 100],
                              labels=['Part-Time', 'Full-Time', 'Overtime'])

edu_map = {
    'Preschool': 'Low', '1st-4th': 'Low', '5th-6th': 'Low', '7th-8th': 'Low',
    '9th': 'Low', '10th': 'Low', '11th': 'Low', '12th': 'Low',
```

Figure 12: Feature engineering for the input payload

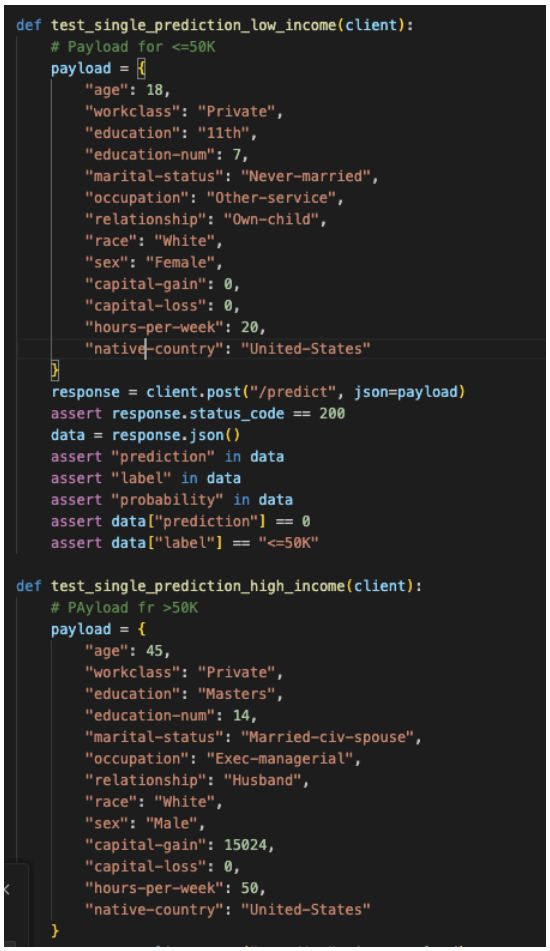
Prediction Types

- **Real Time**: This is ideal for the places where they needs smooth user experience like web forms and loan application portals.
- **Batch**: For usecases where it does not need realtime for example scoring a database of large number of users overnight for target mailings.API Tests

We have implemented a test suites for api validations whech is meant to local and CI level execution.

Currently It has Following Verification cases and figure 13 depicts the test cases.

1. Single Prediction (Low Income): Young person with with no capital gains
2. Single Prediction (High Income): Professional with good capital gain and Master level education
3. Batch Prediction: Send profiels as batch array for verification



```python
def test_single_prediction_low_income(client):
    # Payload for <=50K
    payload = {
        "age": 18,
        "workclass": "Private",
        "education": "11th",
        "education-num": 7,
        "marital-status": "Never-married",
        "occupation": "Other-service",
        "relationship": "Own-child",
        "race": "White",
        "sex": "Female",
        "capital-gain": 0,
        "capital-loss": 0,
        "hours-per-week": 20,
        "native-country": "United-States"
    }
    response = client.post("/predict", json=payload)
    assert response.status_code == 200
    data = response.json()
    assert "prediction" in data
    assert "label" in data
    assert "probability" in data
    assert data["prediction"] == 0
    assert data["label"] == "<=50K"

def test_single_prediction_high_income(client):
    # PAyload fr >50K
    payload = {
        "age": 45,
        "workclass": "Private",
        "education": "Masters",
        "education-num": 14,
        "marital-status": "Married-civ-spouse",
        "occupation": "Exec-managerial",
        "relationship": "Husband",
        "race": "White",
        "sex": "Male",
        "capital-gain": 15024,
        "capital-loss": 0,
        "hours-per-week": 50,
        "native-country": "United-States"
    }
```

Figure 13: Test Suite

Clean Architecture & Dependency Injection Implementation

To ensure modularity, separation of concerns, and ease of testing, the API layer is structured according to Clean Architecture patterns, and Dependency Injection (DI) framework were used for handling dependencies which support scalable test case writing without worrying about server connections.

1. **Layer Separation**:
    - **Domain Model**: Validation and data transfer objects (PredictionInput, PredictionResponse, and BatchPredictionResponse) at domain/model/models.py
    - **Domain Interface**: Declares the interface for model serving (IApiServer)
    - **Domain Use Case**: Houses the Core Business logic: IncomePredictionUseCase depicted in figure 14 below.
        1. Pydantic validation entities
        2. Create the Pandas inputs
        3. Feature engineering logic
        4. Invokes the XGBoost
    - **Data Layer**: Implements the concrete API routing and controller logic by satisfying the IApiServer interface (ApiServerImp) Figure 15 below depcits ApiServer implmentation.

```python
class IncomePredictionUseCase:
    def __init__(self, model_dir: str, load_model_usecase: LoadModelUseCase, logger: ILogger):
        self.logger = logger
        self.model_dir = model_dir
        self.model_path = "Q1/outputs/models/xgboost_model.joblib"

        self.model = load_model_usecase.execute(self.model_path)

    def execute(self, inputs: List[PredictionInput]) -> List[PredictionResponse]:
        self.logger.info(f"Exec IncomePredictionUseCase for {len(inputs)} records")

        if self.model is None:
            raise RuntimeError("Model not loaded")

        # Inputs to raw column conversion with Pydantic
        raw_data = [item.model_dump(by_alias=True) for item in inputs]
        df = pd.DataFrame(raw_data)

        # Feature Eng
        df['capital_net'] = df['capital-gain'] - df['capital-loss']

        # We can predict now
        preds = self.model.predict(df)
        proba = self.model.predict_proba(df)[:, 1]

        # Pydantic Format prediction response
```

Figure 14: IncomePredictionUseCase

```python
class ApiServerImp(IApiServer):
    def __init__(self, usecase: IncomePredictionUseCase, logger: ILogger):
        self.usecase = usecase
        self.logger = logger

        self.api = FastAPI(
            title="Adult Income Prediction API",
            description="Api service for real time and batch income predictions using XGBoost with Clean architecture",
            version="1.0.0"
        )

        self._setup_routes()

    def _setup_routes(self) -> None:
        @self.api.post("/predict", response_mode
        def predict(input_data: PredictionInput)
            self.logger.info("Single prediction
            try:
                results = self.usecase.execute([input_data])
                return results[0]
            except Exception as e:
                self.logger.error(f"Prediction error: {str(e)}")
                raise HTTPException(status_code=400, detail=f"Prediction error: {str(e)}")
```

(parameter) input_data: PredictionInput

Go to PredictionInput

input_data

Figure 15: Api Server Implementation

2. **Dependency Injection**:

- Configurations and register dependencies as depicted in figure 16
- They can by dynamically resolved and injected
- Scope were defined as they requires like singleton or factory

```python
class AppContainer(containers.DeclarativeContainer):

    config = providers.Configuration()

    logger: providers.Provider[ILogger] = providers.Singleton(
        LoggerFactory.create,
        logger_type="console",
        name="ML_CW"
    )

    data_loader: providers.Provider[IDataLoader] = providers.Singleton(
        CsvLoaderImp,
        data_dir=config.data.data_dir,
        logger=logger
    )

    # Use Cases for Task 1
    fetch_raw_data_usecase = providers.Factory(
        FetchRawDataUseCase,
        data_loader=data_loader,
        logger=logger
    )

    analyze_target_usecase = providers.Factory(
        AnalyzeTargetUseCase,
        data_loader=data_loader,
        logger=logger
    )

    # Use Cases for Task 2
    data_cleaning_usecase = providers.Factory(
        DataCleaningUseCase,
        data_loader=data_loader,
```

Figure 16: Dependency Injection Container

MLOps Pipeline and Automation

1. Data and Model Versioning, Publishing and Fetching

- **DVC (Data Version Control)**: Git can handle codes fine but it breaks for large files. Here ML model binaries (xgboost_model.joblib) should not tracked by Git. Here we use DVC to track these binaries.

- **GitOps Tracking Protocol**:
  - Model files are git ignored
  - DVC tracks the binary and generates a pointer file which is a unique hash representing that speciffic version of model file as depicted in figure 17

```yaml
outs:
- md5: 3f629bc3e701c6fa50c56a0ac79b73aa
  size: 380506
  hash: md5
  path: xgboost_model.joblib
  %L for Agent
```

Figure 17: Pointer file(xgboost_model.joblib.dvc) created for XGBost classifier model

- The .dvc file added to git where it can find from the dvc repositary later

◊ **DVC Storage**: This is the model storage allowing pushing and fetching of the model using the registred pointer file above. Here we use a local directory as dvc storage but when it goes to Cloud environment we can use a cloud sorage like Amazon S3.

◊ **Programmatic Model Publishing Use Case**: Clean architecture use case (PublishModelUseCase) under task_4 for model tracking. It adds the model to DVCand pushes the binary artifact to the remote storage. Figure 18 depctis the logic written in python.

```
#  DVC add
self.logger.info(f"Running: {dvc_path} add {rel_model_path}")
add_result = subprocess.run(
    [dvc_path, "add", rel_model_path],
    cwd=workspace_root,
    capture_output=True,
    text=True
)
if add_result.returncode != 0:
    self.logger.error(f"Failed to add model to DVC")
    return False
self.logger.info(f"Successfully added model to DVC:\n{add_result.stdout}")

# 2. DVC push
self.logger.info(f"Running: {dvc_path} push {rel_model_path}.dvc")
push_result = subprocess.run(
    [dvc_path, "push", f"{rel_model_path}.dvc"],
    cwd=workspace_root,
    capture_output=True,
    text=True
)
```

Figure 18: Publishing model to defined storage

◊ **Fetching with DVC Apo**

We load DVC pointed model using DVC python api. Here we have a LoadModelFromDvcUseCase which loads the mode from DVC storage as depicted in figure 18 below. This gives following advantages

1. Keep the docker image smaller. Pullying model dynamicall at app start than putting into docker image.
2. Model version control, when we open the model we can specify git tag which serve as the model version in dvc repo as depcited in Figure 19 and bbelow.

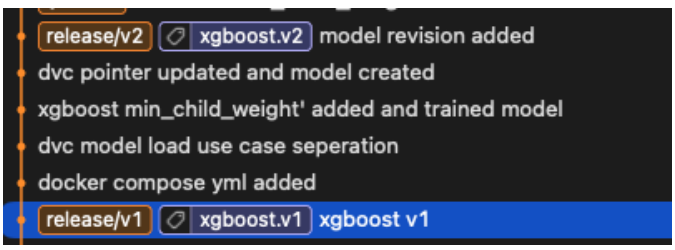


Figure 19a : Git Tag for model versioning

```python
class LoadModelFromDvcUseCase:
    def __init__(self, logger: ILogger):
        self.logger = logger

    def execute(self, model_path: str):
        self.logger.info(f"Loading model via DVC: {model_path}")
        try:
            with dvc.api.open(
                path=model_path,
                mode='rb',
                rev='xgboost.v1'
            ) as model_file:
                model = joblib.load(model_file)
            self.logger.info("Loaded XGBoost model via DVC OK")
            return model
        except Exception as e:
            self.logger.error(f"Failed to load model via DVC: {str(e)}")
            raise e
```

Figure 19b : Load model from DVC storage specifying git tag as the version

Continuous Integration and Continuous Deployment (CI/CD)

We use Buildkite for CI/CD workflows. Figure 20 depicts buildkite steps configuration yaml.

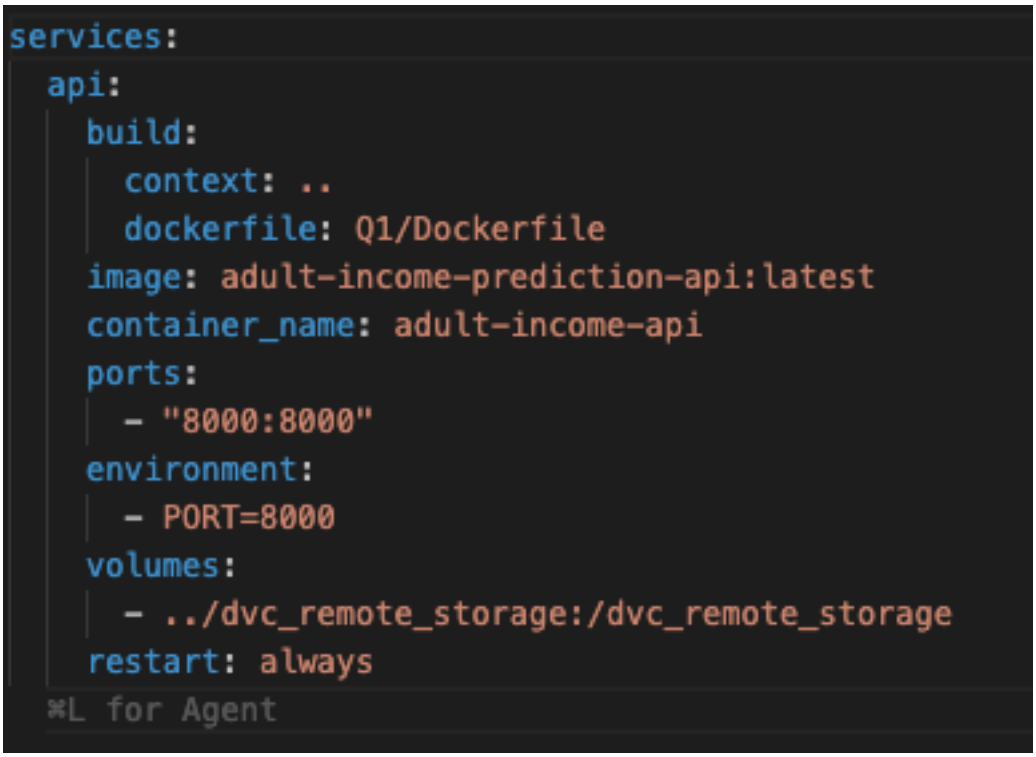
1. **Continuous Integration:** For every Pull Request to git, buildkite triggers the static and unit tests step
2. **Continuous Deployment:** Once the testing step passes buildkite executes the build step. Maintain release branch filterting to execute these stsps.

```yaml
steps:
  - label: ":python: Static analysis and API Unit Tests"
    key: "lint-and-test"
    plugins:
      - docker#v5.9.0:
          image: "python:3.10"
          workdir: "/app"
          environment:
            - "PYTHONPATH=/app/Q1/src"
          command:
            - "sh"
            - "-c"
            - |
              python -m pip install --upgrade pip
              pip install -f Q1/requirements.txt
              flake8 Q1/src --count --select=E9,F63,F7,F82 --show-source --statistics
              # Run pytest
              pytest Q1/src/task_6/tests/

  - wait

  - label: ":docker: Build Docker Image"
    key: "build-image"
    branches: "release/*"
    plugins:
      - docker-build#v1.15.0:
          image-name: "adult-income-prediction-api"
          tag: "latest"
          context: "."
          dockerfile: "Q1/Dockerfile"
```

Figure 20 : Buildkite Steps

Docker Compose – Containerization

We define a local docker environment using docker compose yml as depcited in figure 21. Here we mount he locally hosted remote storage for data version control. Because DVC api can access model cached files for get them inside docker.



```yaml
services:
  api:
    build:
      context: ..
      dockerfile: Q1/Dockerfile
    image: adult-income-prediction-api:latest
    container_name: adult-income-api
    ports:
      - "8000:8000"
    environment:
      - PORT=8000
    volumes:
      - ../dvc_remote_storage:/dvc_remote_storage
    restart: always
  %L for Agent
```

Figure 21: docker-compose yml

Q2 : Advanced Time Series Forecasting

1. Dataset Understanding and Literature Review

BTC-USD Cryptocurrency Dataset

- ◊ Data Source is Yahoo Finance (yfinance API)
- ◊ Daily Frequency
- ◊ 2372 Rows of daily closing price, high, low, open and trading volume

Literature Review

1. **Machine learning model for Bitcoin exchange rate prediction using economic and technology determinants [6]**

   Source: International Journal of Forecasting

   Summary

   Explored Bitcoin forecasting using ARIMA and deep recurrent neural networks (RNN/LSTM). They demonstrated ARIMAis efficient for linear changes but struggeles with extreme vlatality.

2. **Evaluation of Tree-Based Ensemble Machine Learning Models in Predicting Stock Price Direction of Movement [7]**

   Source: Journal of information science and technology, data, knowledge, and communication

   Summary

   Evaluated tree-based ensemble methods (XGBoost) for financial assets. XGBoost is flexible and robust to outliers. Its limited in extrapolating beyond the range of prices observed during training.

3. **A hybrid volatility forecasting framework integrating GARCH, artificial neural network, technical analysis, and principal components analysis [8]**

   Source: Expert Systems with Applications Journal

   Summary

   Proposed a hybrid GARCH+LSTM model. They were used this to prove hybrid statistical and neural models outperforms standalone models.

Model Selection Driven Factors

Cryptocurrency markets are characterized by

1. Non-stationarity : Statistical properties like mean price and variance are constantly shifting, what worked yesterday many not work today.
2. Extreme Volatalitry: Price changes in massive and unpredictable drifts over very short periods
3. Regime Shifts: Market can instatly move from steady upward trend to high-panic spike. Recently oil price hike with Iran war is an example.

Therefore we implement models from different families and evaluate those approaches.

- ◊ **ARIMA :** This is a Statistical model which captures linear autoregressive and moving averages in series
- ◊ **XGBoost :** This is a Machine Learning model leverages a gradient boosting tree structure.
- ◊ **GARCH :** A statistical and econometric model to measure and track market risk to capture volatility.

2. Time Series Exploration and Preprocessing

Preprocessing and Feature Engineering

Thansform raw market data values to structures and model friendly inputs. Figure 2.1 and 2.2 depicts the cleaning and cleaned data set respectively.

- ◊ Remove unnesssary columns like Source
- ◊ Standardizing key column mappings
- ◊ Enforce Numeric and DateTime types
- ◊ Eliminate missing
- ◊ Negate anomalies like like negative values

```python
# Drop source
if 'source' in df.columns:
    df = df.drop(columns=['source'])

# Rename columns
rename_map = {
    'timestamp_utc': 'Date',
    'open_price': 'Open',
    'high_price': 'High',
    'low_price': 'Low',
    'close_price': 'Close',
    'volume': 'Volume'
}
df = df.rename(columns=rename_map)

# Handle Missing Values
initial_missing = df.isnull().sum().sum()
if initial_missing > 0:
    df = df.ffill().bfill()
else:
    self.logger.info("No missing found")

# Convert Date
if 'Date' in df.columns:
    df['Date'] = pd.to_datetime(df['Date'])

# Convert numeric
numeric_cols = ['Open', 'High', 'Low', 'Close', 'Volume']
for col in numeric_cols:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors='coerce')

# handle negative [price,volume] anomalies
numeric_df_cols = df.select_dtypes(include=[np.number]).columns
negative_counts = (df[numeric_df_cols] < 0).sum()
if negative_counts.sum() > 0:
    for col in numeric_df_cols:
        df.loc[df[col] < 0, col] = np.nan
    df = df.ffill().bfill()
    self.logger.info("Negative anomalies handled")
else:
    self.logger.info("No negative found")

# Compute percentage returns for volatility analysis
df['Returns'] = df['Close'].pct_change()
```

Figure 2.1: Cleaning

	Date	Open	High	Low	Close	Volume	Returns
0	2020-01-01	7194.892090	7254.330566	7174.944336	7200.174316	18565664997	0.000000
1	2020-01-02	7202.551270	7212.155273	6935.270020	6985.470215	20802083465	-0.029819
2	2020-01-03	6984.428711	7413.715332	6914.996094	7344.884277	28111481032	0.051452
3	2020-01-04	7345.375488	7427.385742	7309.514160	7410.656738	18444271275	0.008955
4	2020-01-05	7410.451660	7544.497070	7400.535645	7411.317383	19725074095	0.000089

Figure 2.2: Cleaned dataset

Feature Engineering

1. Calendar Features

With the possibility of handing seasonal behaviour but (unlikely associate with crypto trading except weekend trends due to clousure of finacial institutes) we added calendar features.

2. Lag Features

This gives the model a memory. 1: Previos day, 2: 2 days ago. And also it helps to identify cyclic patterns. 7: Weekly patterns, 30: Mothly patterns

```python
def execute(self, df: pd.DataFrame, close_col: str) -> pd.DataFrame:
    self.logger.info("Starting feature engineering")
    df = df.copy()

    # 1. Calendar Features
    df['DayOfWeek'] = df['Date'].dt.dayofweek
    df['DayOfMonth'] = df['Date'].dt.day
    df['Month'] = df['Date'].dt.month
    df['Year'] = df['Date'].dt.year
    df['IsWeekend'] = df['DayOfWeek'].isin([5, 6]).astype(int)

    # 2. Lag Features
    lags = [1, 2, 3, 7, 30]
    for lag in lags:
        df[f'Lag_{lag}'] = df[close_col].shift(lag)
```

Figure 2.2: Feature Engineering

Visualizations

Historical Price Plot

Figure 2.3 depicts BTC historial closing price chart. Bitcoin's supply schedule is hardcoded to cut the mining suppy in half nearly every four years. Historical values depicts exactly same as depicted in figure 2.3 which has occurred in early 2024. This can be identified as macro cycles which is predictable.

Trading Volume Plot

As depicted in figure 2.4 trading volume plots volume spikes align with large price changes which showcasing the capital exchange in bull and bear market.

Figure 2.3: BTC historical closing price

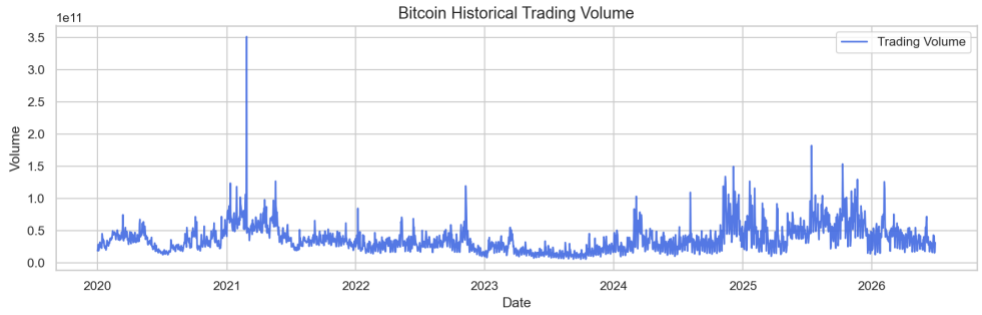


Figure 2.4: Historical Trading Volume

Returns Plot

As depicted in figure 2.5 daily percentage change returns changes around meamn zero. Even BTC changes over time and massive upward trends the percentage changes gives a different picture. On any given date the BTC might go up by 5% and down by 5%. Over a long period all these ups and downds average out ot 0%. Which makes this stationary series goes up and down around a flat baseline.

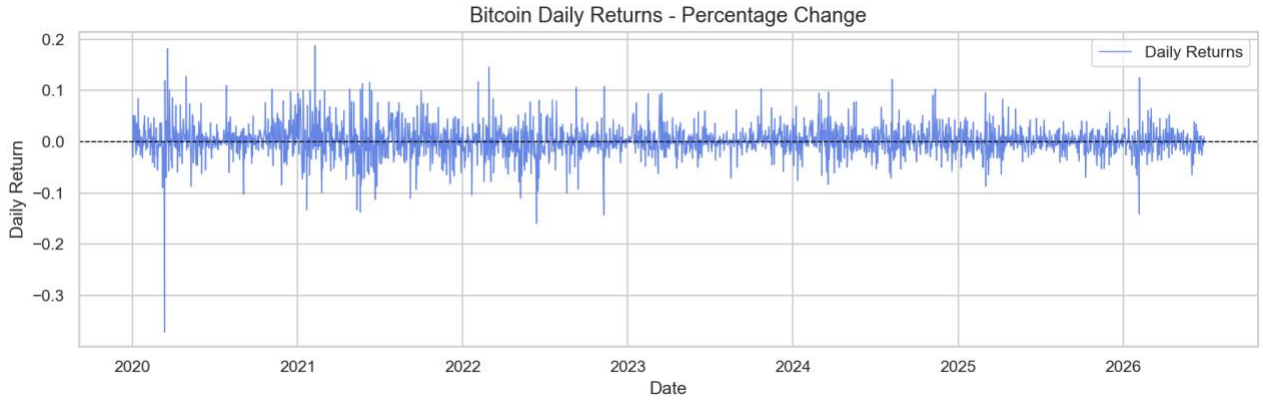


Figure 2.5: Daily Return Plot

Stationarity Tests – ADF(Augmented Dickey Fuller)

ADF evaluates whether time series has a unit root (non stationary and trending) or its stationary.

H0(Null Hypotheis): Series has a unit root, means its non-stationary. (Statistical properties like mean, variance will change over time). This is a one sided test means it simply test greater or less than baseline.

H1(Alternative): It does not have unit root

Here we test close price and daily returns (percentage change) for testing stationary and figure 2.6 depicts ADF test python code and Table 211 illutrates the result.

```python
report = {
    'Test Statistic': result[0],
    'p-value': result[1],
    'Lags Used': result[2],
    'Number of Observations': result[3],
    'Critical Values': result[4]
}

# Significance level is 5%
is_stationary = result[1] <= 0.05

self.logger.info(f"ADF Statistic : {result[0]:.6f}")
self.logger.info(f"p-value : {result[1]:.6e}")
self.logger.info(f"Stationarity Result : {'Stationary' if is_stationary else 'Non-Stationary'} (5% Significance Level)")
```

Figure 2.6: ADF testing python code

Series	**P-val**	**Result (5% Sig Level)**
Close Price	0.5252	Non-Stationary
Returns (Pct Change)	8.96×10^{-29}	**Stationary**

Table 2.1: ADF Test Result

Close Price : P-Val(0.5252) > 0. 05 means it fails to reject null hypothesis
Percentage Change: P-Val(8.96×10^{-29}) < 0.05 means it rejects the null hypothesis, confirms **it's stationary. So this is viable for timeseries and economical modeliing.**

3. Model Development and Optimization

Data Splitting (Train, Validation and Test)

We use **chronological splitting** to avoid data leakages becauses which is a inherant issue of random splitting. For example model may train data from Wednesday and test on Tueday data and train agin from Thursday. Wenesday and Thursday already contain information on what happened on Tuesday. So the model is cheating, predicting the past using knowledge of future.

Here we use **chronological split which past can only predict future not other way.**

- ◊ **Train Set (70%):** 1,639 observations (Jan 31, 2020 to Jul 26, 2024)
- ◊ **Validation Set (15%):** 351 observations (Jul 27, 2024 to Jul 12, 2025)
- ◊ **Test Set (15%):** 352 observations (Jul 13, 2025 to Jun 29, 2026)

Model Implementation

ARIMA

Model Params (p, d, q)

- ◊ P(Autoregressive) : How many past days price history model should look

◊ D(Integration) : How many times the data needs to be differenced to make it stationary

◊ Q(Moving Average): Past forecast error count model should use to smoothening

Grid Search

Instead guessing which valeus for p,d,q are the best, define a grid of possibilities. And it builds the grid of every possible combination.

P = [0, 1, 2]
d = [1]
q = [0, 1, 2]

Here we omit seasonal orders param because bitcoin trades 24 x7 and not much deals with fixed calendar based seasonal cycles. With setting d = 1, the model operates as stationary ARMA (2, 2) on the first differenced absolute price changes of bitcoin.

```python
def tune(self, train_series: pd.Series, p_values=[0, 1, 2], d_values=[1], q_values=[0, 1, 2]) -> tuple:
    self.logger.info("Tuning ARIMA parameters")
    best_aic = float("inf")
    best_order = (1, 1, 1) # Default baseline order

    for p in p_values:
        for d in d_values:
            for q in q_values:
                try:
                    # Omit seasonal_order, crypto not follow seasonal frequencies
                    model = SARIMAX(train_series, order=(p, d, q), enforce_stationarity=False)
                    fit_res = model.fit(disp=False)

                    # Select order with the lowest Akaike Information Criterion (AIC)
                    if fit_res.aic < best_aic:
                        best_aic = fit_res.aic
                        best_order = (p, d, q)
                except:
                    continue

    self.logger.info(f"Best ARIMA: {best_order}")
    return best_order
```

Figure 2.7: ARIMA model implementation

XGBoost

Here the hyperparameters optimized with grid search to minimize RMSE(Root Mean Squared Error). And it gave optimized params as n_estimators: 200, max_depth: 3, and learning_rate: 0.03. Data leakages were prevented by enforcing exclusion of raw price variables(Open, High, Low, Adj Close) and target returns. This ensures model trains on structural indicators and calendar features.

Concerns
Open, High, Low, and Close are highly correlated, if we included them to XGBoost it could bypass engineered feature completely. It may simply look at High or Open value to predict Close calue.

```python
def tune(self, X_train: pd.DataFrame, y_train: pd.Series, X_val: pd.DataFrame, y_val: pd.Series) -> dict:
    # Grid search XGBoost hyperparameters and select lowest validation RMSE.
    self.logger.info("Tuning XGBoost parameters")
    best_rmse = float("inf")
    best_params = {'n_estimators': 100, 'max_depth': 3, 'learning_rate': 0.1}

    # Grid parameters
    learning_rates = [0.03, 0.1]
    max_depths = [3, 5]
    n_estimators = [100, 200]

    for lr in learning_rates:
        for depth in max_depths:
            for nest in n_estimators:
                model = xgb.XGBRegressor(
                    n_estimators=nest,
                    max_depth=depth,
                    learning_rate=lr,
                    random_state=42
                )
                model.fit(X_train, y_train)
                val_preds = model.predict(X_val)
                val_rmse = np.sqrt(mean_squared_error(y_val, val_preds))

                if val_rmse < best_rmse:
                    best_rmse = val_rmse
                    best_params = {
                        'n_estimators': nest,
                        'max_depth': depth,
                        'learning_rate': lr
                    }
```

Figure 2.8: XGBoost model implementation

Model Forecasting Comparison

Following matrix were used for evaluation and results are depicted in Table 2.2

1. Mean Absolute Error (MAE)
2. Root Mean Sqaured Error (RMSE)
3. Mean Absolute Percentage Error (MAPE)

Model	**MAE**	**RMSE**	**MAPE**
ARIMA	**1580.63**	**3370.51**	**1.77%**
XGBoost	23060.47	29707.12	22.23%

Table 2.2: Evalaution Results of Models

Here **ARIMA** significantly outperform XGBoost with MAPE with just 1.77% compared to XGBoost 22.23%

XGBoost has high MAE of ~23000 and RMSE of ~30000. This indicate that it has major directional or scaling shift on the test set.
During training raw prices were entirely removed and XGBoost totally predicts the values from calendar features. Without histroical price fed into XGBoost may lacks the absolute baseline.

Model prediction on test set which depicts in figure 2.9 clearly indicate above behaviour noticed in matrix that ARIMA clearly outperforms XGBoost in test set.

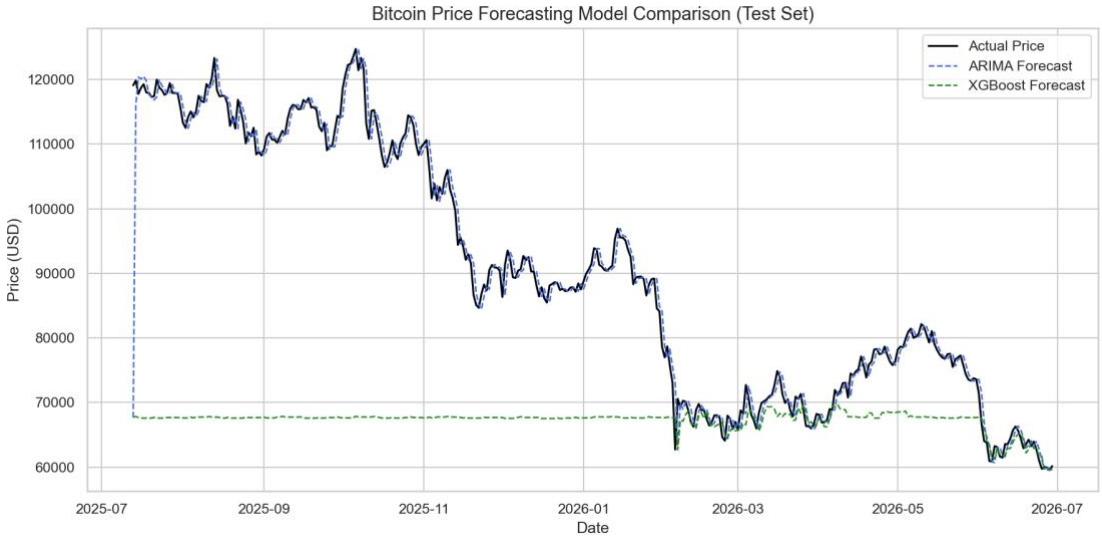


Figure 2.9: Test set model forecasting plots

4. Volatility Modeling using GARCH (Generalized Autoregressive Conditional Heteroskedasticity)

Garch models are exclusively to model and forecase volatality. Below are few concerns with GARCH.

Conditional
Volatality today depends on past information

Heteroskedasticity (Changing variance)
Standard statistical models assumes the market risk profile is same. But changing variance here implies that it is not. The spread of the data(vairance) is constantly changing.

ARCH Effect
This addresses volatality clustering. Current variance is directly related on immediate past events. The visible market outcome is volatality clustering.

1. High variance Condition
   Recently expereince market shock leads to a cluster of large price mevement
2. Low Variance Condition
   Recently it was calm resulting a cluster of small price movement

Due to the above factors in production grade GARCH is not used to predict price direction. Intead it paired with directional models to assess the risk factors.

For an example ***if ARIMA predict price increase, we can use GARCH to forecast volatality. This would help traders to reduce trade size and protect the capital***.

Analyze the Time Series for changing variance

As depicted on figure 2.10 we run ARCH-LM test on returns mean to find the p-value assuming the significance level is 0.05.

```python
def check_arch_effects(self, df: pd.DataFrame, returns_col: str = 'Returns') -> dict:
    self.logger.info("Checking for ARCH effects on returns...")
    returns = df[returns_col].dropna()

    # Run ARCH-LM test on returns mean
    _, p_val, _, _ = het_arch(returns - returns.mean(), maxlag=10)

    self.logger.info(f"ARCH-LM Test done. p-value: {p_val:.6e}")

    # 0.05 is the significance level
    return {
        'p_value': p_val,
        'has_arch_effects': p_val < 0.05
    }
```

Figure 2.10: ARCH effect testing

Here we got significantly low p-value 1.981030×10^{-18} which is below out significance level(0.05). So we can reject null hypothesis of constant variance.

This proves **ARCH effect and volatality clutering present in the BTC dataset.**

Implement the GARCH Model

Here we use GARCH(1,1), p = 1, q = 1

p = Measures impact of recent market shocks on current volatality
q = Persistance of historical volatality over time

p = 1, q = 1, looking 1 step back for both market shock and historical volatality

Figure 2.11 depicts GARCH model (1,1) implementation.

```python
def fit_garch(self, df: pd.DataFrame, returns_col: str = 'Returns', model_path: str = None) -> dict:
    self.logger.info("Fitting GARCH(1,1) model on scaled returns")

    # Scale -> returns x 100 for stability
    returns_scaled = df[returns_col] * 100

    # Constant mean GARCH(1,1) with Normal distribution
    model = arch_model(returns_scaled, mean='Constant', vol='GARCH', p=1, q=1, dist='normal')
    res = model.fit()

    # Parameters
    alpha = res.params['alpha[1]']
    beta = res.params['beta[1]']
    persistence = alpha + beta

    if model_path:
        os.makedirs(os.path.dirname(model_path), exist_ok=True)
        joblib.dump(res, model_path)

    self.logger.info(f"GARCH Fit: alpha={alpha:.4f}, beta={beta:.4f}, persistence={persistence:.4f}")

    return {
        'alpha': alpha, 'beta': beta, 'persistence': persistence
    }
```

Figure 2.11: GARCH Implementation

Alpha	0.0957
Beta	0.8787
Persistance (Alpha + Beta)	0.9744

Table 2.3: GARCH Parameters

ARCH Coeeficient(Alpha)

Measures the immediate impact of yesterday market shock. The value 0.096 satisfies non negative condition. Model is sensitive to short term price shocks.

GARCH Coefficent (Beta)

This measures the impact of historical volatality. Beta is 0.8787 wich is significantly larger than Alpha variance much depends on long term swings rather than short term.

Volatality Persistance

This is 0.9744 which is closer to 1, this means if volatality shock enters it stays longer and decays at very slow rate.

5. Model Comparison, Insights, and Critical Reflection

We already compared ARIMA and XGBoost comparison in above table 2.2.

Critical Reflections

XGBoost failt to comply on Test data set and predicted flat values.

We splits the dataset and assigned 70% for training. But if we refer to the figure 2.3 above which reflects the bitcoin historical data, starting later 2024 bitcoin reached record high which are mostly fits in the test dataset (30%). Trees cannot extrapolate predictions outside the range of target labels observed in training dataset. That is the reason for flat values capped near training maximum which is exactly what addresses in the Ernest et al[7].

Question 3: Optimization using Genetic Algorithms

1. Problem Definition and Literature Review

Project Portfolio Selection (Resource Allocation) was selected as our optimization problem. Here need to select optimal subset of projects from pool of avialable projects to maximize the profit with **considering capital budget constraints, constraints with avialable developer hours and other logical constraints**.

Resource Constraints : Budget Limit and Developer Hours

Logical Constraints :

1. Mutual Exclusion : Certain projects cannot picked together
2. Dependency : Some projects depends on completion of another

Mathematical Formulation

Here we need to Pick **N** number of projects that maximises the profit.

Objective Function

$$Max \sum_{i=1}^{N} Pr_i \ . \ BD_i$$

$$\sum_{i=1}^{N} C_i \ . \ BD_i \leq \text{TB}$$

$$\sum_{i=1}^{N} H_i \ . \ BD_i \leq \text{TDH}$$

Pr_i : Profit of project i
BD_i : Binary Decision Variable $\in (0, 1)$ of selecting the project/not
C_i : Cost of project i
H_i : Hours of project I required
TB : Total Budget Avaialable
TH : Total Developer Hours Avaialable

Literature Review

1. **Revisiting the Evolution of Genetic Algorithms in Scheduling [9]**

   Source: Applied Soft Computing, Elsevier

   Summary

   Conducted a review of nearly 1500 publications on Genetic Algorithms in scheduling. The study showed that GA remains one of the most widely used optimization techniques due to its flexibility and effectiveness.

2. **Matheuristics: Survey and Synthesis [10]**

   Source: International Transactions in Operational Research

   Summary

   Heuristic search and metaheuristics are optimization approaches and here they conducted a study of using Mixed Integer Programming (MIP) with hose techniques. They found that mathematical optimization with heuristic search increase computational efficiency of the solution.

3. **A Didactic Review on Genetic Algorithms for Industrial Planning and Scheduling Problems [11]**

   Source: IFAC-PapersOnLine, Elsevier

   Summary

   Study of using GA for solving industrial and shedulting problems. They found that GA provides high quality near optimal solutions for hard optimization problems while maintaining reasonable computational time.

Slecting GA and MIP

Mixed Integer Programming (MIP)

This is mathematically optimal programming solution to slicing the search space using branch and cut method to make the best possible decision. The “Mixed” part here meant to be handle two distinct types of variables.

1. ***Integer/Binary varaibles*** : This is for problems that cannot be devided into fractions. For example the projet selection above is select or not. We cannot execute the project partially
2. ***Continous variables*** : Numbers may have fraction values. For example the problems like 1.5 devleoper hours if applciable.

The above objective function we have **only one decision variable** and they multiply by known constant, its linear programming problem we are tryingto address. So it’s specialized in linear models we can call it **Mixed Integer Linear Programming**.

Genetic Algorithm (GA)

For simple use cases MIP normally superior because, they execute fast and guarantees we found best possible combination. But when the problem shifts from simple use cases to

more extreme complex use cases GA becomes the preferred choice. Following are some cases where GA becomes the superior choice.

1. Massive problem Size: Eg: 100,000 projects, number of combinations may not be possible for MIP to handle
2. Non-Linear variables: Eg: Profile = Project 1 ^ 2, MIP may fail here, GA do not care linearity
3. Complex Time-sequence dependencies: Eg: due to a holiday season in 2 months developer hours availability significantly reduce.

2. Data Preparation and Problem Analysis

We have generated dataset of size 30 (N = 30) projects as sample shown in figure 3.1 below.

Project	Profit ($K)	Cost ($K)	Dev Hours Required
Enterprise Migration	161.2	65.6	147.2
Enterprise Redesign	230.0	143.3	53.5
Enterprise Integration	184.2	113.8	40.3
Enterprise Automation	194.5	95.8	88.5
Enterprise Dashboard	163.0	36.1	158.0
Enterprise Platform	113.1	36.1	53.9
Enterprise API	108.9	22.8	114.1
Enterprise Pipeline	248.6	131.9	126.6
Enterprise Analytics	181.8	96.2	53.3
Enterprise Refactor	215.5	110.6	122.0

Figure 3.1: Projects

Total Cost for 30 projects: $2226K
Total Developer Hours Required: 2566

We set 50% as the capacity threshold
Total Budget: $1113K
Total Developer Hours Limit: 1283

Mutual Exclusion: From below pairs only 1 project can be selected

1. P3 (Enterprise Automation), P7 (Enterprise Pipeline)
2. P12 (*Enterprise Testing*), P18 (*Enterprise Optimizer*)
3. P22 (*Cloud Integration*), P27 (*Cloud Pipeline*)

Dependency Projects: Below shows interdependent projects.

1. P8 (*Enterprise Analytics*) requires P2 (*Enterprise Integration*)
2. P15 (*Enterprise Portal* requires P10 (*Enterprise Rewrite*)
3. P25 (*Cloud Platform*) requires P20 (*Cloud Migration*)

Figure 3.2 illustrates the data preparation step implementation in python code from use case "DataPreparationUseCase"

```
projects = [Project(id=i, name=names[i], profit=float(profits[i]), cost=float(costs[i]), dev_hours=float(dev_hours[i])) for i in range(num_p

# Set org capacity to 50% of total projects required
total_cost = sum(p.cost for p in projects)
budget = float(np.round(total_cost * 0.50, 1))
total_dev_hours = sum(p.dev_hours for p in projects)
dev_hours_limit = float(np.round(total_dev_hours * 0.50, 1))

self.logger.info(f"Loaded {num_projects} projects. Total Cost: {total_cost:,.1f}k, Total Dev Hours: {total_dev_hours:,.1f}h")
self.logger.info(f"Organization Capacity: Budget: {budget:,.1f}k, Dev Hours limit: {dev_hours_limit:,.1f}h")

# Load constraints
df_const = pd.read_csv(c_path)
mutual_exclusions, dependencies = [], []

for _, row in df_const.iterrows():
    a, b = int(row['project_a']), int(row['project_b'])
    if a < num_projects and b < num_projects:
        # Mutual exclusion: A or B
        if row['constraint_type'] == "mutual_exclusion":
            mutual_exclusions.append((a, b))
        # Depend: pick B if A is chosen
        elif row['constraint_type'] == "dependency":
            dependencies.append((a, b))

self.plot_proj_distribution(costs, profits, dev_hours, num_projects, plot_dir)

return AllocationInstance(
    projects=projects,
    budget=budget,
    dev_hours_limit=dev_hours_limit,
    mutual_exclusions=mutual_exclusions,
    dependencies=dependencies
)
```

Figure 3.2 Data Preparation implementation

Figure 3.3 illustrates project distribution plots which visualize the cost vs profit tradeoff. The size of the points represents the required developer dev hours.

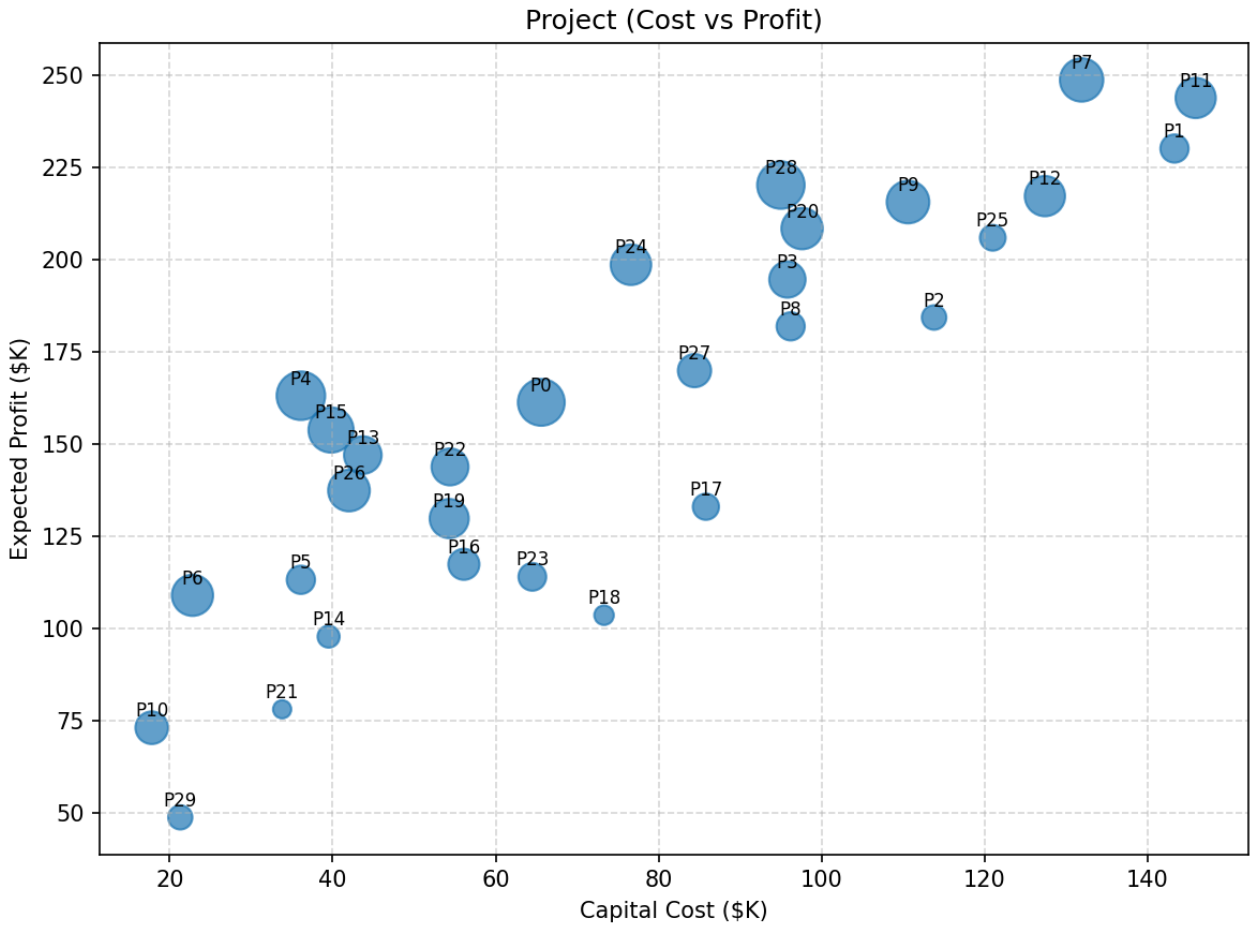


Figure 3.3: Project Distribution Plot

From the figure 3.3 plot, we can say best projects sit in the top-left region where high profit and low cost and we need to consider about smaller point size which is low developer hours required.

Best Candidates

- ◊ P4, P15, P13 and P26: High profit for low cost
- ◊ P6: Decent profit for very low cost

High Profit but Cost or Labor requirement is intensive

- ◊ P7, P11, P1: Highest Profitable projects but requires heavy capital cost to incur
- ◊ P28: Very good profit but large point which means high developer hours consumption

Projects to Avoid

- ◊ P18, P17: Locates in the bottom-right region, lower profit and high capital cost
- ◊ P29: Very low profit

Complexity Analysis

The size of the search space for 30 projects is 2^{30} which is nearly 1.07 x 10^9 possible combinations. Brute force evaluation of over 1 billion combinations is computationally very expensive which requires better solutions like GA and MIP.

3. Genetic Algorithm Design and Implementation

Chromosome representation and Popualtion Initalization Strategy

Becase we have 30 projects we need to created a binary array of size 30 as the representaion of a chromasome. This is same as the feature select/not usecase we use binary represenation to keep feature or drop as the chromasome. As the same in the projet selectrion usecase of ours we need to decide keep the project or not.

$$Chromasome = [1 = keep\ the\ project, 0 = not, \dots]$$

The population size and generation count are dynamically configurable to support hyperparameter tuning as depicted in figure 3.4

```python
# uning_results = ga_solver_usecase.hyperParamTune(instance, pop_sizes=[50, 100, 200, 300], gens=[50, 100, 150, 200])
def hyperParamTune(self, instance: AllocationInstance, pop_sizes: List[int], gens: List[int]) -> Dict[str, Any]:
    """
    GA and hyper param tuning with dynamic pop size and generation number
    """
    best_profit = -1.0
    best_params = {}
    N = len(instance.projects)
    results = []
    for pop_size in pop_sizes:
        for gen in gens:
            population = [list(np.random.randint(0, 2, N)) for _ in range(pop_size)]
```

Figure 3.4 : Population creation

Fitness Function

We have fitness function implemented with score based on total expected profit. For the fitness scores we assign low and high penalties for vialations.

Low Penalties : Vialations of Budget and Labour constraints assign low penalties.

High Penalties : Logical constraint vialation like mutual exclusion and dependency we assign high penalty which brings negative scores. Here we have picked 500 which exceed maximum profit from the 30 pool projects.

Finally fitness value calculated as below function and logics depicted in figure 3.5 below.

$$Fitness\ Value = Total\ Profit - Penalty$$

```python
total_profit = 0.0
total_cost = 0.0
total_labor = 0.0
violation_count = 0

for idx, val in enumerate(chromosome):
    if val == 1:
        p = instance.projects[idx]
        total_profit += p.profit
        total_cost += p.cost
        total_labor += p.dev_hours

# mutual exclusion constraints
for a, b in instance.mutual_exclusions:
    if chromosome[a] == 1 and chromosome[b] == 1:
        violation_count += 1

# dependency constraints
for a, b in instance.dependencies:
    if chromosome[a] == 1 and chromosome[b] == 0:
        violation_count += 1

# resource constraints penalties
penalty = 0.0
if total_cost > instance.budget:
    penalty += 10.0 * (total_cost - instance.budget)
if total_labor > instance.dev_hours_limit:
    penalty += 10.0 * (total_labor - instance.dev_hours_limit)

# High penalty for logical vialation, 500 - this is greater than the profit of all single project (100 to 250)
penalty += 500.0 * violation_count

fitness = total_profit - penalty # could be negative but ok for tounament selection
```

Figure 3.5: Fitness function for GA

Selection Method (Tournament selection)

This is used to select the fittest candidate as a parent from the current population to crossover to produce childrens for next generation as depciuted in figure 3.6 below.

```python
best_idx = indices[0]
for idx in indices[1:]:
    if fitnesses[idx] > fitnesses[best_idx]:
        best_idx = idx
return population[best_idx].copy()
```

Figure 3.6: Tounament selection implementation

Crossover Operators

This is combining two parent chromasomes to create a child. The idea behind is child inherits the best traits. Here we apply uniform cross over mechanism. Its like coin is tossed and determines whether the parent 1 and parent 2 properties are swapped and create and update child 1 and child 2 properties which is the cross over. The logic is depicted in figure 3.7 below.

Additionally we do not cross over 100% instead we keep a cross over rate to 80% which make sure there is a 20% chance that parent pass traits starigh to the next generation without being traits swapped. This make sure good solutions from tournament selection survives intact into

the next generation witohut breakdown in every generation. If 100% crossover we cannot make sure this would preserve.

```python
def crossover(self, parent1: List[int], parent2: List[int]) -> Tuple[List[int], List[int]]:
    """
    mixing 2 parenets -> create 2 children, child inherits best traits. uses uniform corss(coin toss)
    """
    child1 = parent1.copy()
    child2 = parent2.copy()
    for i in range(len(parent1)):
        if np.random.rand() < 0.5:
            child1[i], child2[i] = parent2[i], parent1[i] # crossed
    return child1, child2
```

Figure 3.7 : Unform Crossover

Mutation Operation

Random Small changes introduced to prevent algorithm stuck in a local trap. Logic is depicted in figure 3.8 below.

$$Before = [0, 1, 0 \dots]$$

$$After = [0, 0, 1 \dots \ ]$$

```python
def mutate(self, chromosome: List[int], mutation_rate: float) -> List[int]:
    """
    Small Random changes to prevent algo stuck in local trap
    """
    mutated = chromosome.copy()
    for i in range(len(mutated)):
        if np.random.rand() < mutation_rate:
            mutated[i] = 1 - mutated[i]  # flip the bit (1->0 or 0->1)
    return mutated
```

Figure 3.8: Random mutation with bit flipping

Elitism

This is a process of automatically copying the best chromosomes directly into the next generation without crossover or mutation. Without Elitism we would notice a oscilation as depicted in figure 3.9. With addressing elitism of selecting the best score for next generation

its continously improved and stabalize as depicted in figure 3.10 below. Figure 3.11 below depicts elitism implementation.

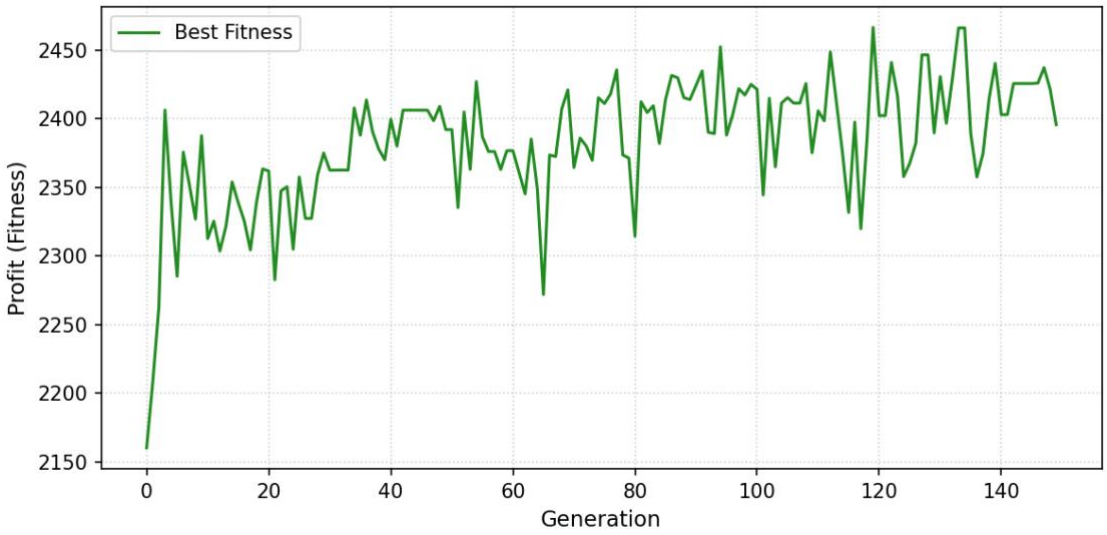


Figure 3.9: GA Best Score Curve without Elitism (Up and Down)

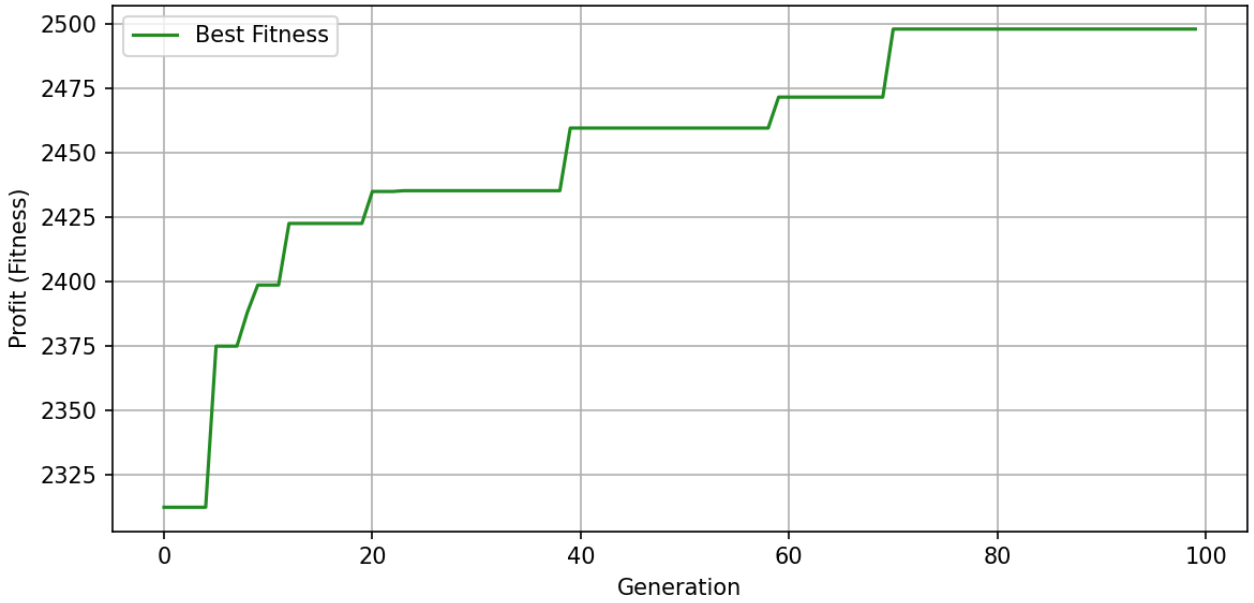


Figure 3.10: GA Best Score Curve with Elitism

```python
# next generation, (with/without elitism)
# new_population = self.elitism(population, fitnesses) # []

def elitism(self, population: List[List[int]], fitnesses: List[float]) -> List[List[int]]:
    """
    Copying best individual to next generation, without we wont see continous improvement, but up and down
    """
    best_idx = np.argmax(fitnesses)
    return [population[best_idx].copy()]
```

Figure 3.11 : Elitism implementation

Hyperparameter tuning

The main parameters we use for GA here are population size and generations size. To identify the best parameters for 30 project portfolios, we used the param value combination as

Population Size $\in \{50, 100, 200, 300\}$

Generations $\in \{\mathbf{50}, \mathbf{100}, \mathbf{150}, \mathbf{200}\}$

After searching over the grid space depicted in table 3.1 below, the **best parameters** found that maximized the profit is

Population Size: 50
Generations: 100
Profit: $2497K

	Generation 50	**Generation 100**	**Generation 150**	**Generation 200**
Population 50	(50, 50)	(50, 100)	(50, 150)	(50, 200)
Population 100	(100, 50)	(100, 100)	(100, 150)	(100, 200)
Population 150	(150, 50)	(150, 100)	(150, 150)	(150, 200)
Population 200	(200, 50)	(200, 100)	(200, 150)	(200, 200)

Table 3.1: Grid Space for population and Generation

Results

From the 30 projects portfolio we were able to pick the optimum portfolio as illustrated in figure 3.12. The result is 16 projects have been selected and 14 got rejected with maximum profit gain with maximum resource allocation as depicted in table 3.2 below.

Maximum Profit	$2497.2
Cost	Spent $1108 from Budget $1113.2K
Dev Hours	Used 1275h from 1283.3h available

Table 3.2: BILP execution result for project 30

Figure 3.12 below depicts the optimum solution picked by GA. For 30 projects portfolio it has picked 15 projects.

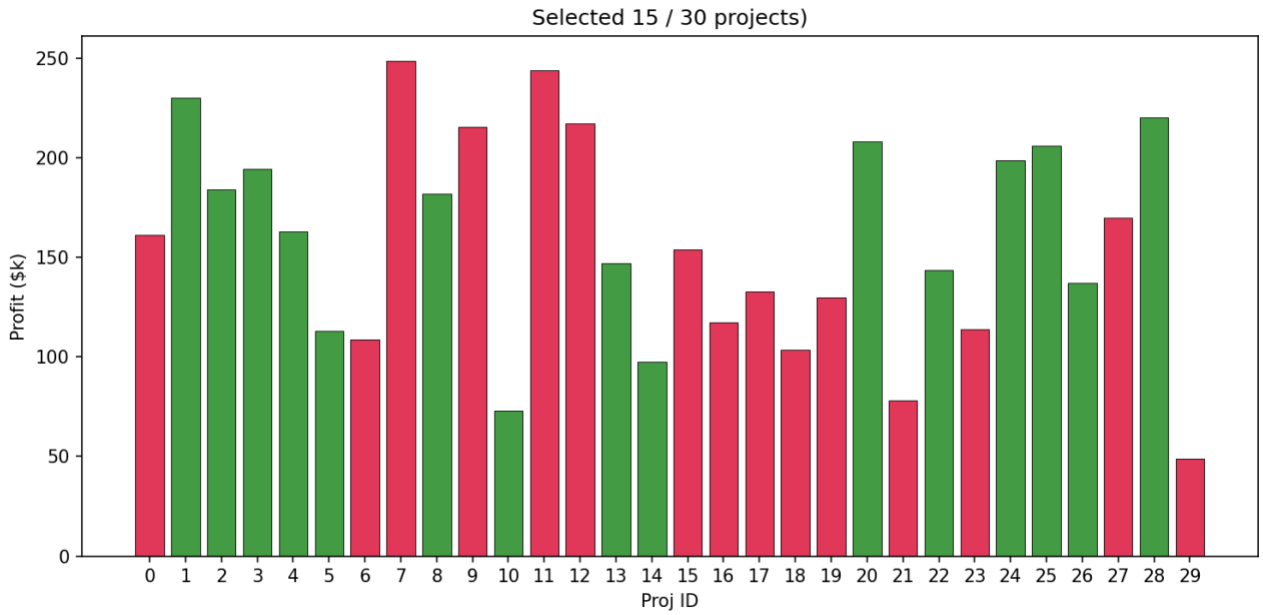


Figure 3.12: GA results of portfolio selection (Green – selected, Red, Rejected)

4. Mixed Integer Programming

As we addressed in the section 1 above, we are trying to address Binary Integer Linear Programming Model here where decision variable is zero or one.

Binary Decision Variable (BD)

$$BD = \begin{cases} 1 \\ 0 \end{cases}$$

1: Project I selected
0: Project I did not select

Objective Function (F)

$$Maximize \sum_{i=1}^{N} Pr_i \,.\, BD_i$$

Pr_i : Profit from project i
BD: Binary decision of project (i) (1 or 0)

Constraints

As above we have resource constraints and logical constraints.

Resource Constraints

1. Total required hours for all selected projects should be less than or equal to Total Developer Hours available
2. Total required cost for all selected projects should be less than or equal to Total Budget willing to spent

Logical Constraints

1. Mutual Exclusion, where if 2 projects conflicting only one of them could pick
2. Dependency, where selecting of a project may require another project already selected

Implement the MIP model

Figure 3.13 below illustrates our implementation of BILP using PuLP python lib. Unlike GA this guarantees to finding the single optimum project portfolio.

```python
def find_best_projects(self, instance: AllocationInstance) -> Tuple[AllocationSolution, float]:
    """
    MIP with PuLP (BILP here)
    """
    # Problem is find Max profit which is the optimization here
    prob = pulp.LpProblem("Project Portfolio Selection to Maximize Profit", pulp.LpMaximize)

    # binary decision var to select/not project
    N = len(instance.projects)
    x = pulp.LpVariable.dicts("project_select", range(N), cat=pulp.LpBinary)

    # function is to sum up profit
    prob += pulp.lpSum([x[i] * p.profit for i, p in enumerate(instance.projects)])

    # Budget limit constraint
    prob += pulp.lpSum([x[i] * p.cost for i, p in enumerate(instance.projects)]) <= instance.budget, "Budget_Limit"

    # dev hors limit constraint
    prob += pulp.lpSum([x[i] * p.dev_hours for i, p in enumerate(instance.projects)]) <= instance.dev_hours_limit, "Dev_Hours_Limit"

    # Setup logical constraints
    # Mutex
    for idx, (a, b) in enumerate(instance.mutual_exclusions):
        prob += x[a] + x[b] <= 1, f"Mutual_Exclusion_{idx}"

    # Dep
    for idx, (a, b) in enumerate(instance.dependencies):
        prob += x[a] <= x[b], f"Dependency_{idx}"

    status = prob.solve()
```

Figure 3.13: BILP implementation

Results

From the 30 projects portfolio we were able to pick the optimum portfolio as illustrated in figure 3.14 below. The result is 16 projects have been selected and 14 got rejected with maximum profit gain with maximum resource allocation as depicted in table 3.3 below.

Maximum Profit	$2511.7K
Cost	Spent $1109.2K from Budget $1113.2K
Dev Hours	Used 1281.4h from 1283.3h available
Execution Time	0.21 seconds

Table 3.3: BILP execution result for project 30

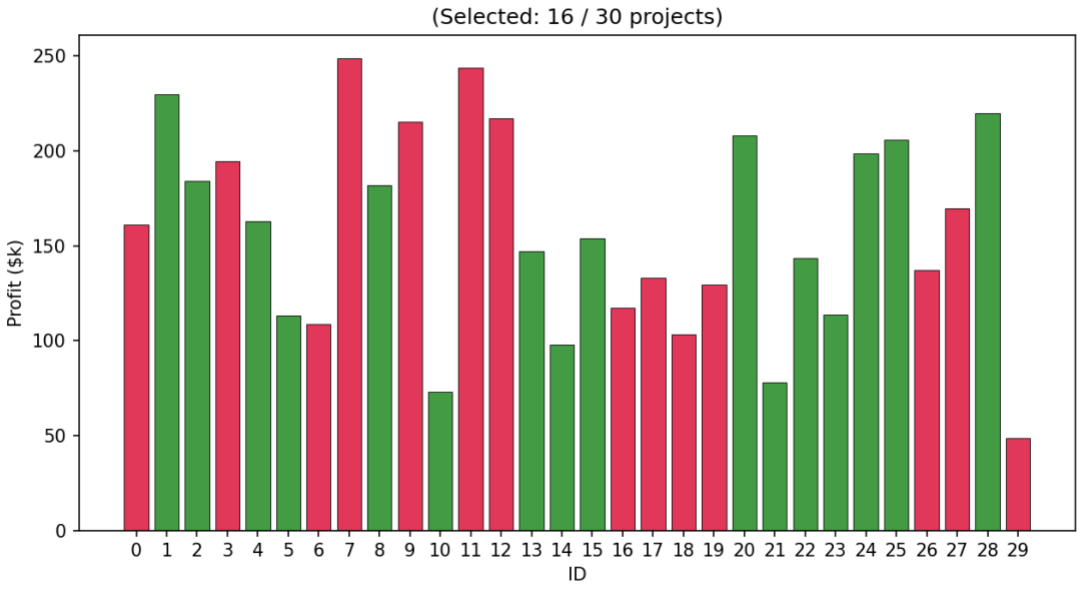


Figure 3.14 BILP(MIP) results of portfolio selection (Green – selected, Red, Rejected)

5. Comparative Analysis and Critical Reflection

We used project size range as [10, 20, 50, 100, 150, 300, 500, 600, 700, 800, 900, 1000] for comparing GA and MIP. We have 30 projects data in our original data set and for additional project we create the data with random values and use the same resource and logical constraints above.

Finally, we plot the execution time against the project size as the figure 3.15 below.

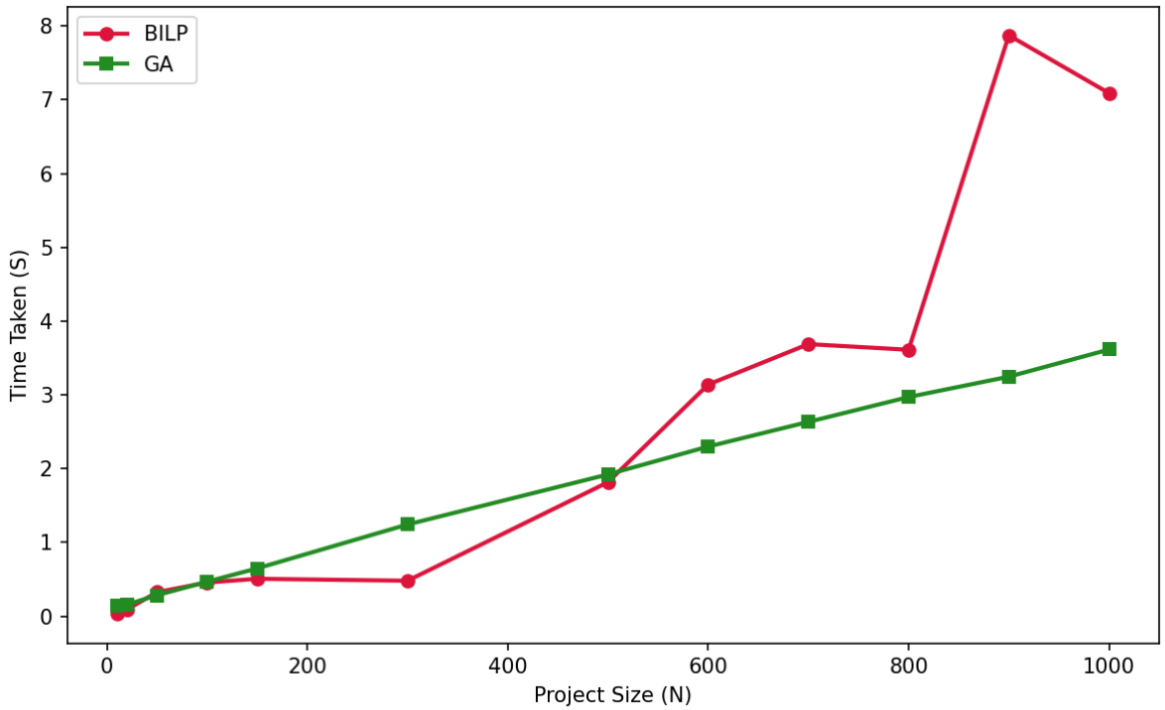


Figure 3.15: GA vs BILP/MIP (Execution time against Project size)

Comparison

Optimul Solution

	Profit ($K)	Cost ($K)	Dev Hours	Project Count
GA	2497	1108	1275	15
BILP (MIP)	2511	1109	1281	16

Table 3.4: BA vs BILP optimul solution

MIP algorithm gurantees the absolute optimum but GA finds a near optimual solution. GA profit is 99.4% of BILP which is almost identical.

Computation Time

As depicted in figure 3.15 above for small project sizes (<500) BILP/MIP is faster. For larger problems MIP becomes slower due to greater level of branch nodes and GA time scales linearly. MIP time is highly volatile but GA time is prdictable due to linear scaling.

References

1. R. Kohavi, "Scaling up the accuracy of Naive-Bayes classifiers: A decision-tree hybrid," in *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD)*, Portland, OR, USA, 1996, pp. 202–207.
2. A. Chockalingam et al., "Comparative study of machine learning algorithms on census income data set," *International Journal of Engineering Research and Applications (IJERA)*, vol. 9, no. 8, pp. 78–81, Aug. 2019.
3. M. Topiwalla, "Machine learning on UCI Adult dataset using various classifier algorithms and scaling up the accuracy using extreme gradient boosting," *Technical Report*, Feb. 2017. [Online]. Available: WordPress.com.
4. L. B. Sena and J. C. Machado, "Evaluation of fairness in machine learning models using the UCI Adult Dataset," in *Proceedings of the Brazilian Symposium on Databases (SBBD)*, 2024, pp. 1–6.
5. C. Dewi, R. A. Andika, E. Haryani, D. Riantama, A. Sajid, M. M. Alam, and M. M. Su'ud, "Feature selection for financial data classification using Random Forest, Boruta, and Recursive Feature Elimination," *Ingénierie des Systèmes d'Information*, vol. 30, no. 8, pp. 2165–2173, Aug. 2025. doi: 10.18280/isi.300822.
6. W. Chen, H. Xu, L. Jia, and Y. Gao, "Machine learning model for Bitcoin exchange rate prediction using economic and technology determinants," *International Journal of Forecasting*, vol. 37, no. 1, pp. 28–43, Jan.–Mar. 2021, doi: 10.1016/j.ijforecast.2020.02.008.
7. S. Dey, Y. Roy, and S. Das, "Evaluation of tree-based ensemble machine learning models in predicting stock price direction of movement," *Information*, vol. 11, no. 6, p. 332, Jun. 2020. doi: 10.3390/info11060332.
8. W. Kristjanpoller and M. C. Minutolo, "A hybrid volatility forecasting framework integrating GARCH, artificial neural network, technical analysis, and principal components analysis," *Expert Systems with Applications*, vol. 109, pp. 1–11, Nov. 2018. doi: 10.1016/j.eswa.2018.05.011.
9. K.-C. Ying, P. Pourhejazy, and K.-Y. Chen, "Revisiting the evolution of genetic algorithms in scheduling," Applied Soft Computing, vol. 201, Art. no. 115602, Jun. 2026, doi: 10.1016/j.asoc.2026.115602.
10. M. A. Boschetti, A. N. Letchford, and V. Maniezzo, "Matheuristics: Survey and synthesis," *International Transactions in Operational Research*, vol. 30, no. 6, pp. 2840–2866, Nov. 2023, doi: 10.1111/itor.13301.
11. A. Neumann, A. Hajji, M. Rekik, and R. Pellerin, "A didactic review on genetic algorithms for industrial planning and scheduling problems," IFAC-PapersOnLine, vol. 55, no. 10, pp. 2593–2598, 2022, doi: 10.1016/j.ifacol.2022.10.100.